

AWS Solution Architect Associate



Modules

Cloud Computing

AWS introduction

Compute Services

Network services

Storage Services

Database Services

Deployment and Management services

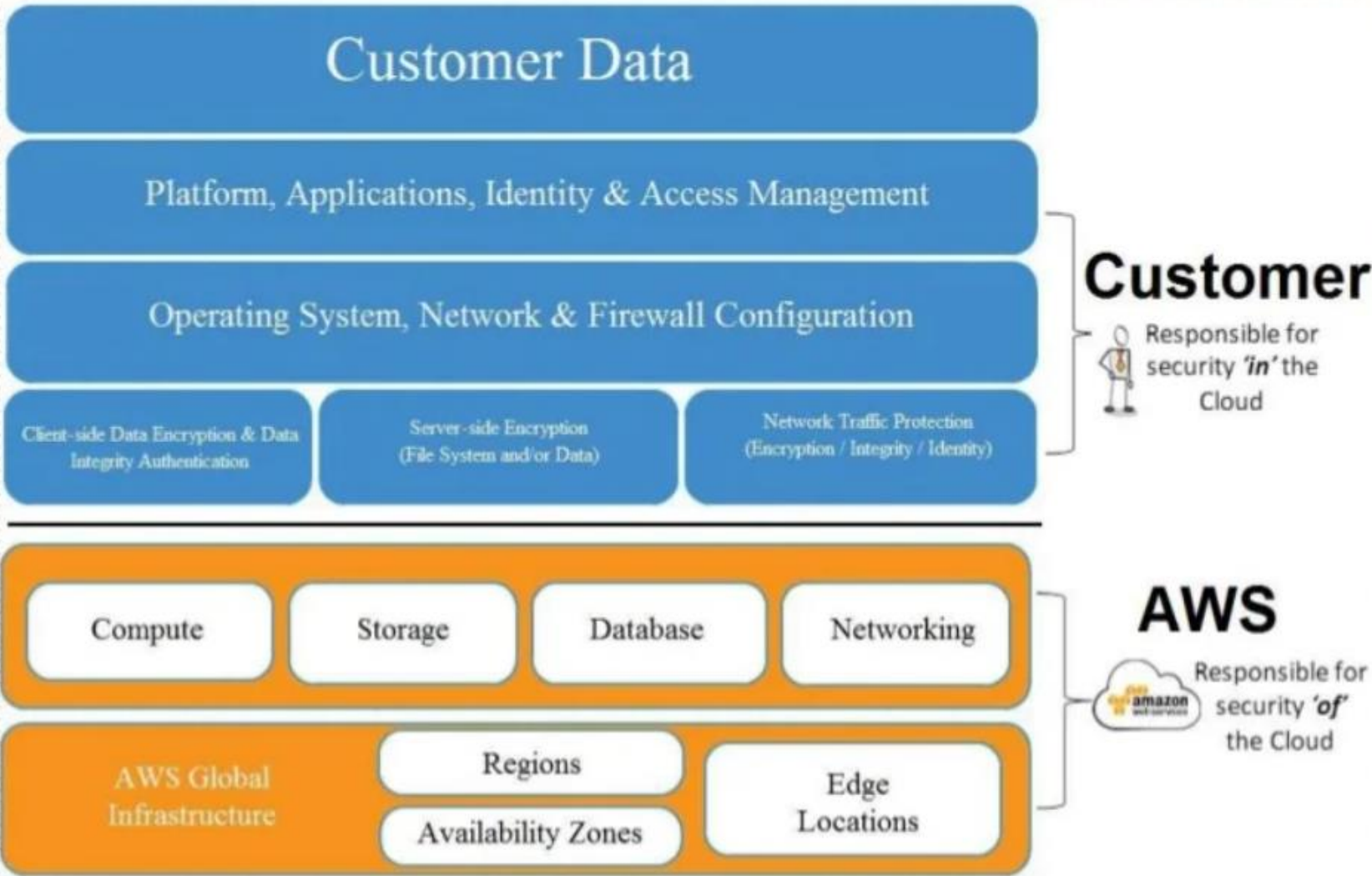
Application and Other services

Security @AWS

- ✓ Identity and Access Management (IAM) to securely control access to users
- ✓ Resource Based policies attached to individual resources like *S3 storage buckets*
- ✓ Network firewalls built into Amazon VPC like *Security groups* and *Subnet ACLs*
- ✓ Secure and private connection options between on-premise and *AWS VPCs*
- ✓ *Web Application Firewall (WAF)* and *AWS Shield* capabilities
- ✓ Encryption at rest for Storage and Database Services
- ✓ *AWS KMS* and *HSM* services for Encryption keys storage and management
- ✓ *Cloud Trail* to log all the API Calls
- ✓ AWS environments are continuously audited, with certifications from accreditation bodies across geographies and verticals
- ✓ The following is a partial list of assurance programs with which AWS complies:
 - SOC 1/ISAE 3402, SOC 2, SOC 3
 - FISMA, DIACAP, and FedRAMP
 - PCI DSS Level 1
 - ISO 9001, ISO 27001, ISO 27018

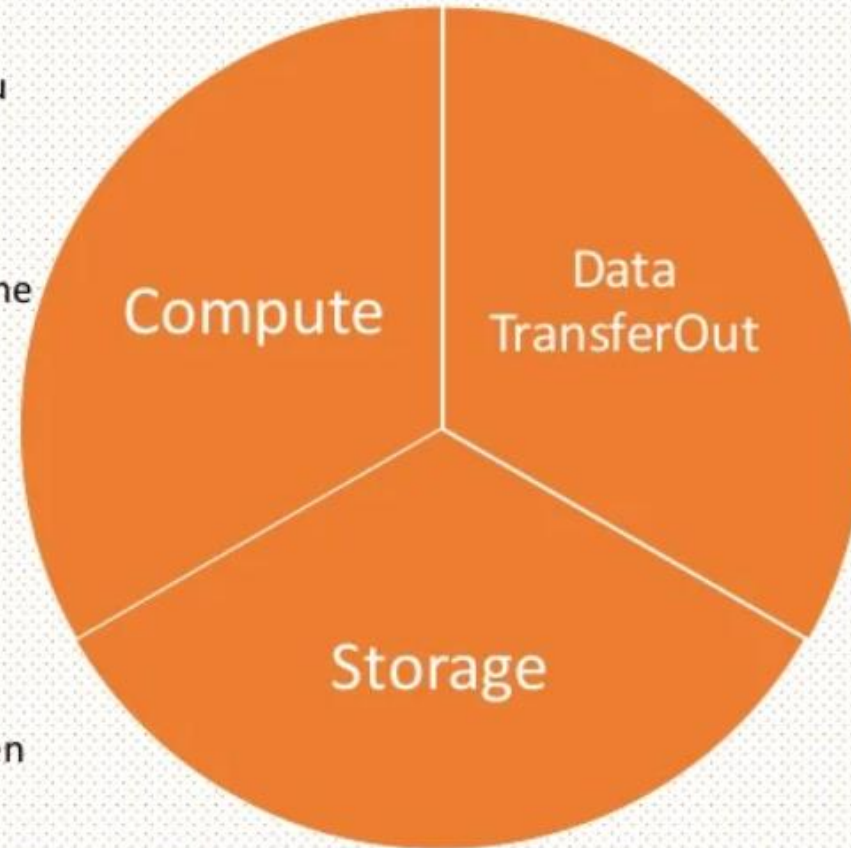


Security @AWS



AWS Pricing Characteristics

- ❖ These characteristics vary slightly depending on the AWS product you are using.
- ❖ However, fundamentally these are the core characteristics that have the greatest impact on cost.
- ❖ There is no charge for inbound data transfer or for data transfer between other Amazon Web Services within the same region
- ❖ The outbound data transfer is aggregated across AWS services and then charged at the outbound data transfer rate



AWS Pricing Philosophy



Pay as you go

Pay less when you reserve

Pay even less per unit by using more

Pay even less as AWS grows

Custom pricing

AWS Free Services

AWS also offers a variety of services for no additional charge:

- ❖ Amazon VPC
- ❖ AWS Elastic Beanstalk
- ❖ AWS CloudFormation
- ❖ AWS Identity and Access Management (IAM)
- ❖ Auto Scaling
- ❖ AWS OpsWorks
- ❖ CloudWatch
- ❖ Many Migration services



AWS Free Tier

The AWS Free Tier enables you to gain free, hands-on experience with the AWS platform, products, and services.



Free Usage Tier
For New AWS Customer

*Free for one year since AWS sign up

**12 months free
products**

Compute

Amazon EC2
750 Hours

per month

STORAGE & CONTENT DELIVERY

Amazon S3
5 GB

of standard storage

Database

Amazon RDS
750 Hours

per month of db.t2.micro

Compute

AWS Lambda
1 Million

free requests per month

Analytics

Amazon QuickSight
1 GB

of SPICE capacity

Simple Monthly Calculator

Whether you are running a single instance or dozens of individual services, You can estimate your monthly bill using AWS Simple Monthly Calculator.

<http://calculator.s3.amazonaws.com/index.html>



Key Resources

Official
Documentation

<https://aws.amazon.com/documentation/>

White Papers

<https://aws.amazon.com/whitepapers/>

News blogs

<https://aws.amazon.com/blogs/aws/>

FAQ

<https://aws.amazon.com/faqs/>

Official
Youtube Channel

<https://www.youtube.com/user/AmazonWebServices>

Certification Roadmap



Role-Based Certifications

Specialty Certifications



Compute Services

EC2

Autoscaling

Elastic load balancer

Elastic Beanstalk

AWS Lambda

EC2

- EC2 Features
- Amazon Machine Images
- Instances
- Monitoring
- Networking and Security
- Storage
- Placement Groups
- T2 instances
- Status Checks



EC2 Features

- Virtual computing environments, known as *instances*
- Preconfigured templates for your instances, known as *Amazon Machine Images (AMIs)*, that package the bits you need for your server (including the operating system and additional software)
- Various configurations of CPU, memory, storage, and networking capacity for your instances, known as *instance types*
- Secure login information for your instances using *key pairs* (AWS stores the public key, and you store the private key in a secure place)
- Storage volumes for temporary data that's deleted when you stop or terminate your instance, known as *instance store volumes*
- Metadata, known as *tags*, that you can create and assign to your Amazon EC2 resources

EC2 features (Contd..)

- Persistent storage volumes for your data using Amazon Elastic Block Store (Amazon EBS), known as *Amazon EBS volumes*
- Multiple physical locations for your resources, such as instances and Amazon EBS volumes, known as *regions and Availability Zones*
- A firewall that enables you to specify the protocols, ports, and source IP ranges that can reach your instances using *security groups*
- Static IPv4 addresses for dynamic cloud computing, known as *Elastic IP addresses*
- Virtual networks you can create that are logically isolated from the rest of the AWS cloud, and that you can optionally connect to your own network, known as *virtual private clouds (VPCs)*

Amazon Machine Images (AMI)

An AMI provides the information required to launch an instance, which is a virtual server in the cloud.

You must specify a source AMI when you launch an instance

An AMI includes the following:

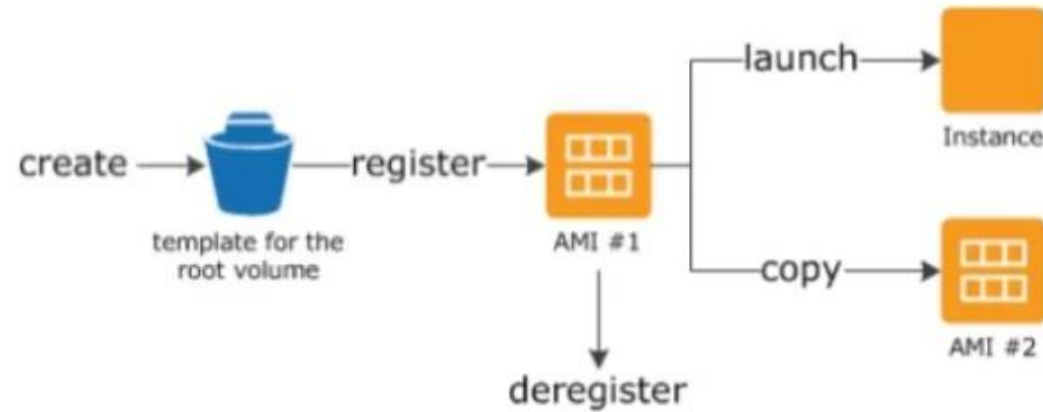
A template for the root volume for the instance (for ex, an operating system, an application server, and applications)

Launch permissions that control which AWS accounts can use the AMI to launch instances

A block device mapping that specifies the volumes to attach to the instance when it's launched

AMI Life cycle

- After you create and **register an AMI**, you can use it to launch new instances
- You can also launch instances from an AMI if the AMI owner grants you launch permissions.
- You can copy an AMI within the same region or to different regions.
- When you no longer require an AMI, you can **deregister it**.



AMI Types

You can select an AMI to use based on the following characteristics:

- Region (see Regions and Availability Zones)
- Operating system
- Architecture (32-bit or 64-bit)
- Launch Permissions
- Storage for the Root Device

Launch Permissions

- The owner of an AMI determines its availability by specifying **launch permissions**.
- Launch permissions fall into the following categories:

- The owner grants launch permissions to all AWS accounts

Public

- The owner grants launch permissions to specific AWS accounts

Explicit

- The owner has implicit launch permissions for an AMI.

Implicit

EC2 Root Device Volume

When you launch an instance, the root device volume contains the **image** used to boot the instance.

You can choose between AMIs backed by Amazon EC2 **instance store** and AMIs backed by **Amazon EBS**.

AWS recommend that you use AMIs backed by Amazon EBS, because they **launch faster and use persistent storage**.

Instance Store Backed Instances:

- Instances that use **instance stores** for the root device automatically have one or more instance store volumes available, with one volume serving as the root device volume
- The data in instance stores is **deleted when the instance is terminated** or if it fails (such as if an underlying drive has issues).
- Instance store-backed instances **do not support the Stop action**
- After an instance store-backed instance fails or terminates, it cannot be restored.
- If you plan to use Amazon EC2 instance store-backed instances
 - distribute the data on your instance stores across multiple Availability Zones
 - back up critical data on your instance store volumes to persistent storage on a regular basis

EBS Backed Instances:

- Instances that use *Amazon EBS* for the root device automatically have an Amazon EBS volume attached
- An Amazon EBS-backed instance **can be stopped** and later restarted without affecting data stored in the attached volumes.
- There are various instance and volume-related tasks you can do when an Amazon EBS-backed instance is in a stopped state.

For example, you can modify the properties of the instance, you can change the size of your instance or update the kernel it is using, or you can attach your root volume to a different running instance for debugging or any other purpose

Instance Types

When you launch an instance, the instance type that you specify determines the hardware of the host computer used for your instance.

Each instance type offers different compute, memory, and storage capabilities and are grouped in instance families based on these capabilities

Amazon EC2 dedicates some resources of the host computer, such as CPU, memory, and instance storage, to a particular instance.

Amazon EC2 shares other resources of the host computer, such as the network and the disk subsystem, among instances.

Available Instance Types

General Purpose : T2 , M5

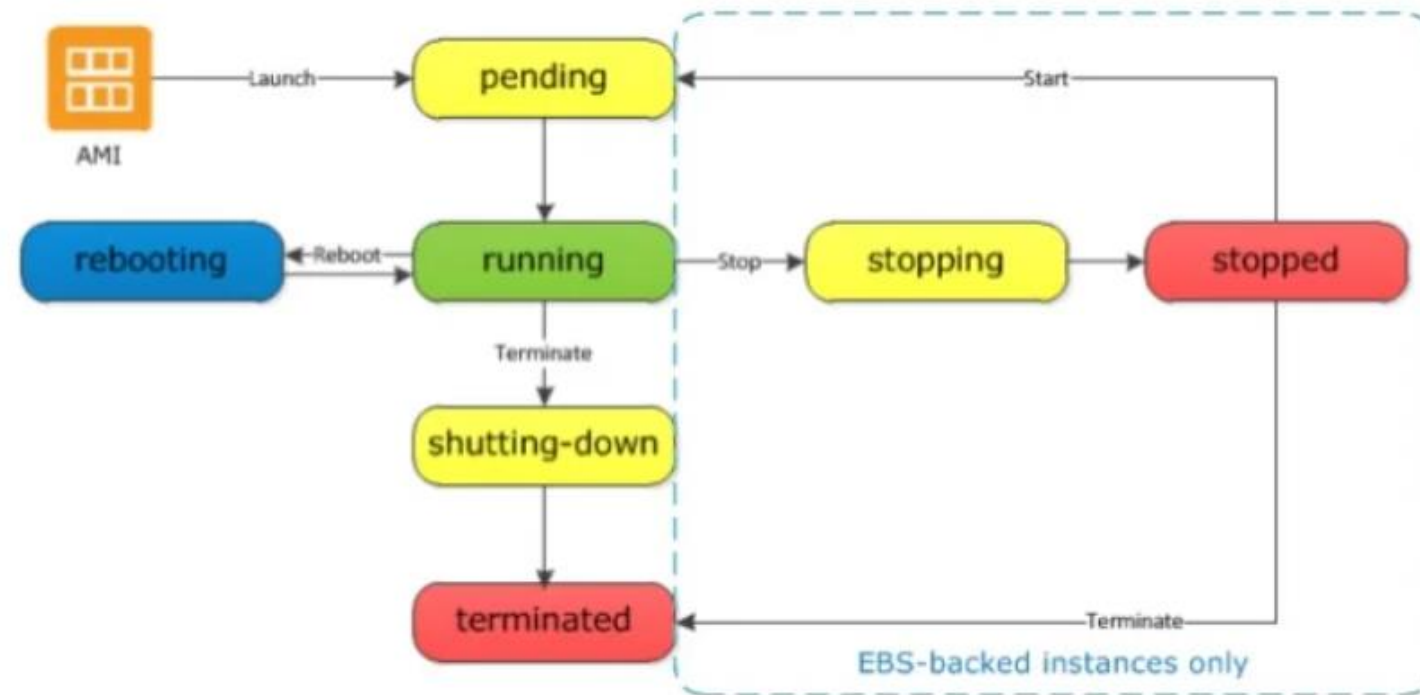
Compute Optimised : C5

Memory Optimized : R4, X1

Storage Optimised: D2, H1, I3

Accelerated Computing: F1, G3, P3

Instance Lifecycle



On-Demand Instances – Pay, by the second, for the instances that you launch.

Reserved Instances – Purchase, at a significant discount, instances that are always available, for a term from one to three years

Scheduled Instances – Purchase instances that are always available on the specified recurring schedule, for a one-year term.

Spot Instances – Request unused EC2 instances, which can lower your Amazon EC2 costs significantly.

Dedicated Hosts – Pay for a physical host that is fully dedicated to running your instances, and bring your existing per-socket, per-core, or per-VM software licenses to reduce costs.

Dedicated Instances – Pay, by the hour, for instances that run on single-tenant hardware.

Instance Purchasing Options

Security Groups

A security group acts as a virtual firewall that controls the traffic for one or more instances.

When you launch an instance, you associate one or more security groups with the instance.

You add rules to each security group that allow traffic to or from its associated instances

When you specify a security group as the source or destination for a rule, the rule affects all instances associated with the security group

SG Rules

- For each rule, you specify the following:
 - **Protocol**: The protocol to allow. The most common protocols are 6 (TCP) 17 (UDP), and 1 (ICMP).
 - **Port range** : For TCP, UDP, or a custom protocol, the range of ports to allow. You can specify a single port number (for example, 22), or range of port numbers
 - **Source or destination**: The source (inbound rules) or destination (outbound rules) for the traffic.
 - **(Optional) Description**: You can add a description for the rule; for example, to help you identify it later.

By default, security groups allow all outbound traffic.

You can't change the outbound rules for an EC2-Classic security group.

Security group rules are always permissive; you can't create rules that deny access.

Security groups are stateful — if you send a request from your instance, the response traffic for that request is allowed to flow in regardless of inbound security group rules.

You can add and remove rules at any time. Your changes are automatically applied to the instances associated with the security group after a short period

When you associate multiple security groups with an instance, the rules from each security group are effectively aggregated to create one set of rules to determine whether to allow access

SG Rules Characteristics

Instance IP addressing

- Every instance is assigned with IP addresses and IPv4 DNS hostnames by AWS using DHCP
- Amazon EC2 and Amazon VPC support both the IPv4 and IPv6 addressing protocols
- By default, Amazon EC2 and Amazon VPC use the IPv4 addressing protocol; you can't disable this behavior.
- Types Of IP addresses available for EC2:
 - Private IP4 addresses
 - Public V4 addresses
 - Elastic IP addresses
 - IPV6 addresses



Private IPV4 addresses

A private IPv4 address is an IP address that's not reachable over the Internet.

You can use private IPv4 addresses for communication between instances in the same network

When you launch an instance, AWS allocate a primary private IPv4 address for the instance from the subnet

Each instance is also given an internal DNS hostname that resolves to the primary private IPv4 address

A private IPv4 address remains associated with the network interface when the instance is stopped and restarted, and is released when the instance is terminated

Public IPV4 addresses

A public IP address is an IPv4 address that's reachable from the Internet.

You can use public addresses for communication between your instances and the Internet.

Each instance that receives a public IP address is also given an external DNS hostname

A public IP address is assigned to your instance from Amazon's pool of public IPv4 addresses, and is not associated with your AWS account

You cannot manually associate or disassociate a public IP address from your instance

Public IP Behavior

- You can control whether your instance in a VPC receives a public IP address by doing the following:
 - Modifying the public IP addressing attribute of your subnet
 - Enabling or disabling the public IP addressing feature during launch, which overrides the subnet's public IP addressing attribute
- In certain cases, AWS release the public IP address from your instance, or assign it a new one:
 - when an instance is stopped or terminated. Your stopped instance receives a new public IP address when it's restarted.
 - when you associate an Elastic IP address with your instance, or when you associate an Elastic IP address with the primary network interface (eth0) of your instance in a VPC.

An Elastic IP address is a static IPv4 address designed for dynamic cloud computing

An Elastic IP address is associated with your AWS account.

With an Elastic IP address, you can mask the failure of an instance or software by rapidly remapping the address to another instance in your account

An Elastic IP address is a public IPv4 address, which is reachable from the internet

By default, all AWS accounts are limited to five (5) Elastic IP addresses per region, because public (IPv4) internet addresses are a scarce public resource

Elastic IP addresses

To use an Elastic IP address, you first allocate one to your account, and then associate it with your instance or a network interface

You can disassociate an Elastic IP address from a resource, and reassociate it with a different resource

A disassociated Elastic IP address remains allocated to your account until you explicitly release it

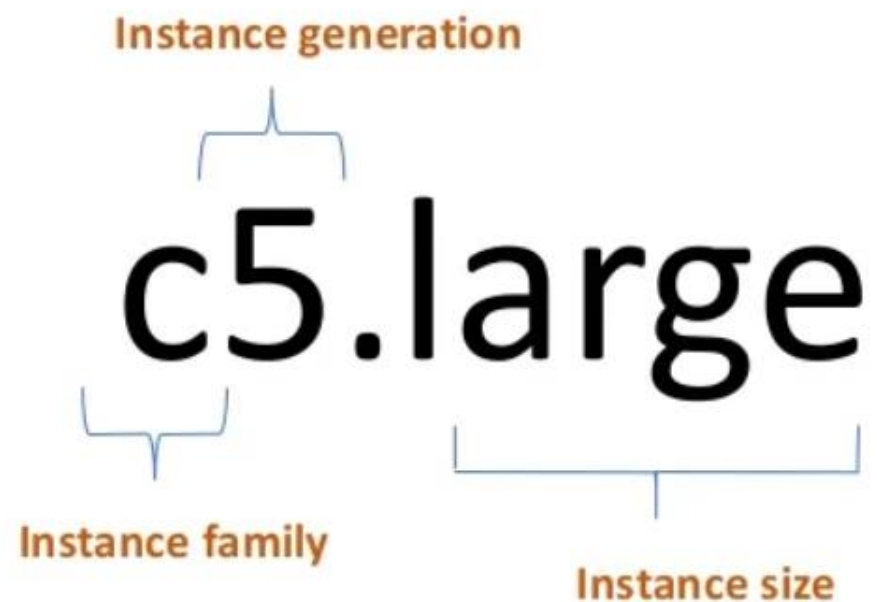
AWS impose a small hourly charge if an Elastic IP address is not associated with a running instance, or if it is associated with a stopped instance or an unattached network interface

While your instance is running, you are not charged for one Elastic IP address associated with the instance, but you are charged for any additional Elastic IP addresses associated with the instance

An Elastic IP address is for use in a specific region only

Elastic IP characteristics

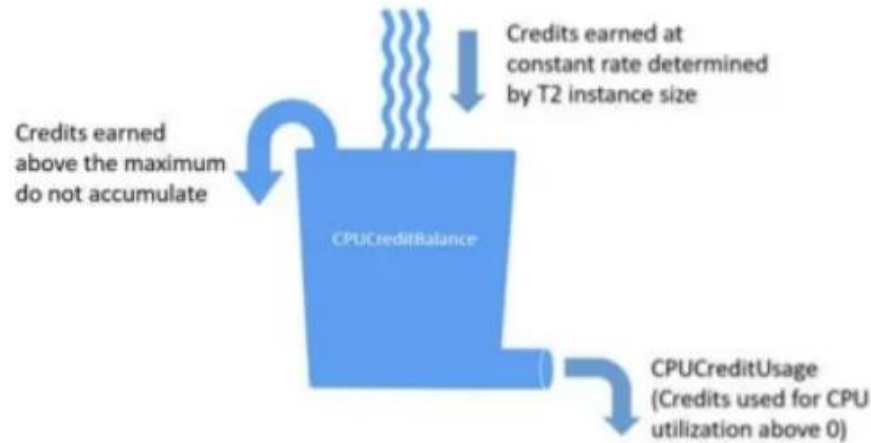
Instance naming



T2 Instances

- T2 instances are designed to provide a *baseline level* of CPU performance with the ability to burst to a higher level when required by your workload
- There are two types of T2 instance offerings : 1 . T2 standard and 2. T2 Unlimited.
- T2 Standard is the default configuration; if you do not enable T2 Unlimited, your T2 instance launches as Standard.
- The baseline performance and ability to burst are governed by *CPU credits*
- A T2 Standard instance receives two types of CPU credits: earned credits and launch credits
- When a T2 Standard instance is in a running state, it continuously earns a set rate of earned credits per hour
- At start, it has not yet earned credits for a good startup experience; therefore, to provide a good startup experience, it receives launch credits at start
- The number of accrued launch credits and accrued earned credits is tracked by the CloudWatch metric *CPUCreditBalance*.
- One CPU credit is equal to one vCPU running at 100% utilization for one minute.
- T2 Standard instances get 30 launch credits per vCPU at launch or start. For example, a t2.micro has one vCPU and gets 30 launch credits, while a t2.xlarge has four vCPUs and gets 120 launch credits

CPU Credit Balance



- If a T2 instance uses fewer CPU resources than is required for baseline performance , the unspent CPU credits are accrued in the CPU credit balance
- If a T2 instance needs to burst above the baseline performance level, it spends the accrued credits
- The number of CPU credits earned per hour is determined by the instance size
- While earned credits never expire on a running instance, there is a limit to the number of earned credits an instance can accrue
- Once the limit is reached, any new credits that are earned are discarded
- CPU credits on a running instance do not expire. However, the CPU credit balance does not persist between instance stops and starts

T2 Unlimited

- T2 Unlimited is a configuration option for T2 instances that can be set at launch, or enabled at any time for a running or stopped T2 instance.
- T2 Unlimited instances can burst above the baseline for as long as required
- This enables you to enjoy the low T2 instance hourly price, and ensures that your instances are never held to the baseline performance.
- If a T2 Unlimited instance depletes its CPU credit balance, it can spend *surplus credits* to burst beyond the baseline
- If the average CPU utilization of an instance is at or below the baseline, the instance incurs no additional charges, Because an instance earns a maximum number of credits in a 24-hour period
- However, if CPU utilization stays above the baseline, the instance cannot earn enough credits to pay down the surplus credits it has spent.
- The surplus credits that are not paid down are charged at a flat additional rate per vCPU-hour
- T2 Unlimited instances do not receive launch credits.

You can change the size of your instance to fit the right workload or take advantages of features of new generation instances.

If the root device for your instance is an EBS volume, you can change the size of the instance simply by changing its instance type, which is known as resizing it.

If the root device for your instance is an instance store volume, you must migrate your application to a new instance with the instance type that you need

You can resize an instance only if its current instance type and the new instance type that you want are compatible with features like virtualization type , kernel type etc.

We can take Instance-store backed AMI in order to migrate instances with instance store root volumes.

Changing Instance Type

Status checks

- Amazon EC2 performs automated checks on every running EC2 instance to identify hardware and software issues.
- This data augments the utilization metrics that Amazon CloudWatch monitors (CPU utilization, network traffic, and disk activity).
- Status checks are performed every minute and each returns a pass or a fail status. If all checks pass, the overall status of the instance is OK.
- If one or more checks fail, the overall status is impaired.
- Status checks are built into Amazon EC2, so they cannot be disabled or deleted.
- You can, however create or delete alarms that are triggered based on the result of the status checks
- There are two types of status checks: system status checks and instance status checks.

System status checks

- Monitor the AWS systems on which your instance runs.
- These checks detect underlying problems with your instance that require AWS involvement to repair
- When a system status check fails, you can choose to wait for AWS to fix the issue, or you can resolve it yourself.
- For instances backed by Amazon EBS, you can stop and start the instance yourself, which in most cases migrates it to a new host computer.
- For instances backed by instance store, you can terminate and replace the instance.
- The following are examples of problems that can cause system status checks to fail:
 - Loss of network connectivity
 - Loss of system power
 - Software issues on the physical host
 - Hardware issues on the physical host that impact network reachability

Instance Status Checks

- Monitor the software and network configuration of your individual instance.
- These checks detect problems that require your involvement to repair.
- When an instance status check fails, typically you will need to address the problem yourself
- The following are examples of problems that can cause instance status checks to fail:
 - Failed system status checks
 - Incorrect networking or startup configuration
 - Exhausted memory
 - Corrupted file system
 - Incompatible kernel

Placement Groups

You can launch or start instances in a placement group, which determines how instances are placed on underlying hardware.

When you create a placement group, you specify one of the following strategies for the group:

- Cluster—clusters instances into a low-latency group in a single Availability Zone
- Spread—spreads instances across underlying hardware

Cluster placement Group

- A cluster placement group is a logical grouping of instances within a single Availability Zone.
- Placement groups are recommended for applications that benefit from low network latency, high network throughput, or both.
- launch the number of instances that you need in the placement group in a single launch request and that you use the same instance type for all instances in the placement group.
- If you receive a capacity error when launching an instance in a placement group that already has running instances, stop and start all of the instances in the placement group, and try the launch again.
- Restarting the instances may migrate them to hardware that has capacity for all the requested instances.

Spread Placement Group

A spread placement group is a group of instances that are each placed on distinct underlying hardware.

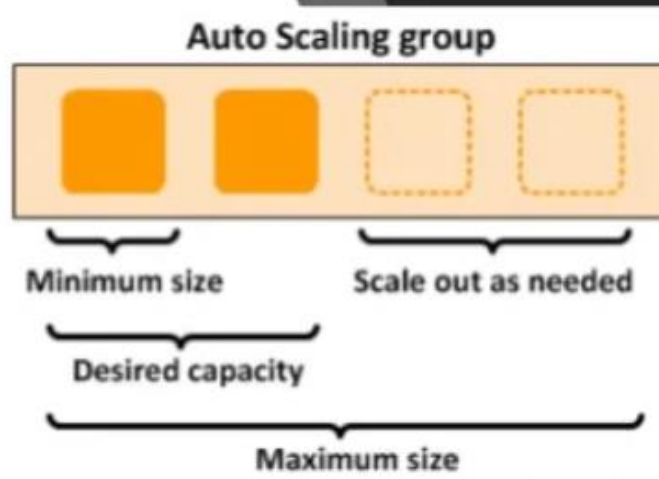
Spread placement groups are recommended for applications that have a small number of critical instances that should be kept separate from each other

Launching instances in a spread placement group reduces the risk of simultaneous failures that might occur when instances share the same underlying hardware.

Spread placement groups provide access to distinct hardware, and are therefore suitable for mixing instance types or launching instances over time.

A spread placement group can span multiple Availability Zones, and you can have a maximum of seven running instances per Availability Zone per group.

Auto Scaling



- You create collections of EC2 instances, called **Auto Scaling groups**.
- You can specify the minimum number of instances in each Auto Scaling group, and Auto Scaling ensures that your group never goes below this size
- You can specify the maximum number of instances in each Auto Scaling group, and Auto Scaling ensures that your group never goes above this size
- If you specify the desired capacity, either when you create the group or at any time thereafter, Auto Scaling ensures that your group has this many instances.
- If you specify scaling policies, then Auto Scaling can launch or terminate instances as demand on your application increases or decreases.

Auto Scaling Components



Groups:

Your EC2 instances are organized into *groups* so that they can be treated as a logical unit for the purposes of scaling and management.



Launch configurations:

Your group uses a *launch configuration* as a template for its EC2 instances. When you create a launch configuration, you can specify information such as the AMI ID, instance type, key pair, security groups, and block device mapping for your instances



Scaling plans:

A *scaling plan* tells Auto Scaling when and how to scale. For example, you can base a scaling plan on the occurrence of specified conditions (dynamic scaling) or on a schedule.

Benefits of Auto scaling

Better fault tolerance

- Auto Scaling can detect when an instance is unhealthy, terminate it, and launch an instance to replace it. You can also configure Auto Scaling to use multiple Availability Zones.

Better availability

- Auto Scaling can help you ensure that your application always has the right amount of capacity to handle the current traffic demand

Better cost management

- Auto Scaling can dynamically increase and decrease capacity as needed. Because you pay for the EC2 instances you use, you save money by launching instances when they are actually needed and terminating them when they aren't needed

Auto Scaling attempts to distribute instances evenly between the Availability Zones that are enabled for your Auto Scaling group

Auto Scaling does this by attempting to launch new instances in the Availability Zone with the fewest instances.

After certain actions occur, your Auto Scaling group can become unbalanced between Availability Zones.

Auto Scaling compensates by rebalancing the Availability Zones.

When rebalancing, Auto Scaling launches new instances before terminating the old ones, so that rebalancing does not compromise the performance or availability of your application

Instance Distribution

Auto Scaling Lifecycle

The EC2 instances in an Auto Scaling group have a path, or lifecycle, that differs from that of other EC2 instances

The lifecycle starts when the Auto Scaling group launches an instance and puts it into service

The lifecycle ends when you terminate the instance, or the Auto Scaling group takes the instance out of service and terminates it.

Life Cycle : Scale Out

- The following scale out events direct the Auto Scaling group to launch EC2 instances and attach them to the group:
 - You manually increase the size of the group
 - You create a scaling policy to automatically increase the size of the group based on a specified increase in demand
 - You set up scaling by schedule to increase the size of the group at a specific time.

Life Cycle : Scale In

- It is important that you create a corresponding scale in event for each scale out event that you create.
- The Auto Scaling group uses its termination policy to determine which instances to terminate.
- The following scale in events direct the Auto Scaling group to detach EC2 instances from the group and terminate them:
 - You manually decrease the size of the group
 - You create a scaling policy to automatically decrease the size of the group based on a specified decrease in demand.
 - You set up scaling by schedule to decrease the size of the group at a specific time.

Instances In Service

Instances remain in the InService state until one of the following occurs:

- A scale in event occurs, and Auto Scaling chooses to terminate this instance in order to reduce the size of the Auto Scaling group.
- You put the instance into a Standby state.
- You detach the instance from the Auto Scaling group.
- The instance fails a required number of health checks, so it is removed from the Auto Scaling group, terminated, and replaced

Attach an Instance

You can attach a running EC2 instance that meets certain criteria to your Auto Scaling group. After the instance is attached, it is managed as part of the Auto Scaling group.

Detach an Instance

You can detach an instance from your Auto Scaling group. After the instance is detached, you can manage it separately from the Auto Scaling group or attach it to a different Auto Scaling group.

LifeCycle Hooks : Launch

You can add a lifecycle hook to your Auto Scaling group so that you can perform custom actions when instances launch or terminate.

The instances start in the Pending state.

If you added an `autoscaling:EC2_INSTANCE_LAUNCHING` lifecycle hook to your Auto Scaling group, the instances move from the Pending state to the Pending:Wait state

After you complete the lifecycle action, the instances enter the Pending:Proceed state.

When the instances are fully configured, they are attached to the Auto Scaling group and they enter the InService state

LifeCycle Hooks : Terminate

When Auto Scaling responds to a scale in event, it terminates one or more instances.

These instances are detached from the Auto Scaling group and enter the Terminating state

If you added an `autoscaling:EC2_INSTANCE_TERMINATING` lifecycle hook to your Auto Scaling group, the instances move from the Terminating state to the Terminating:Wait state.

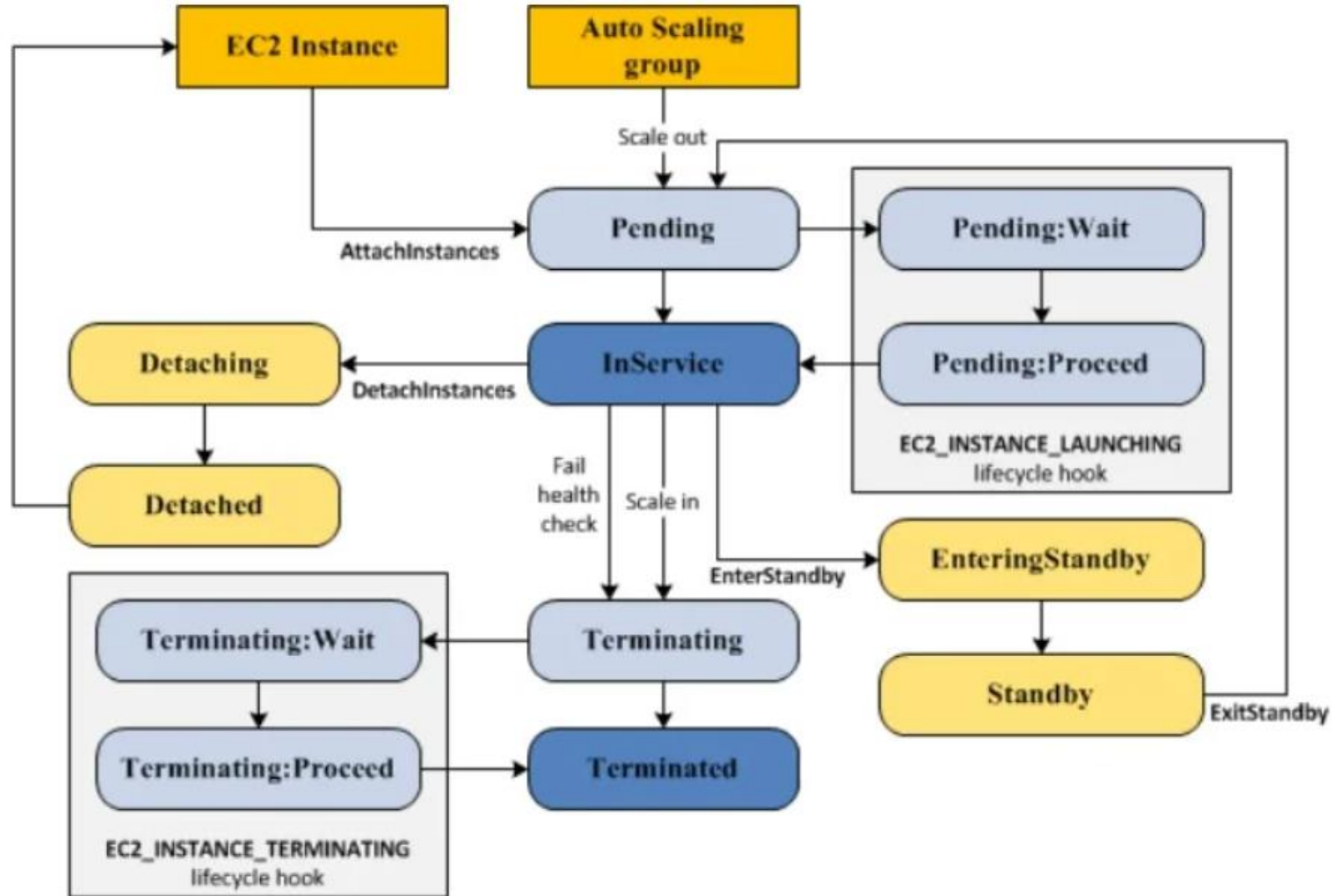
After you complete the lifecycle action, the instances enter the Terminating:Proceed state.

Enter and Exit Standby

- You can put any instance that is in an InService state into a Standby state.
- This enables you to remove the instance from service, troubleshoot or make changes to it, and then put it back into service
- Instances in a Standby state continue to be managed by the Auto Scaling group. However, they are not an active part of your application until you put them back into service.



Auto Scale Life Cycle



Health Checks for Auto Scaling Instances

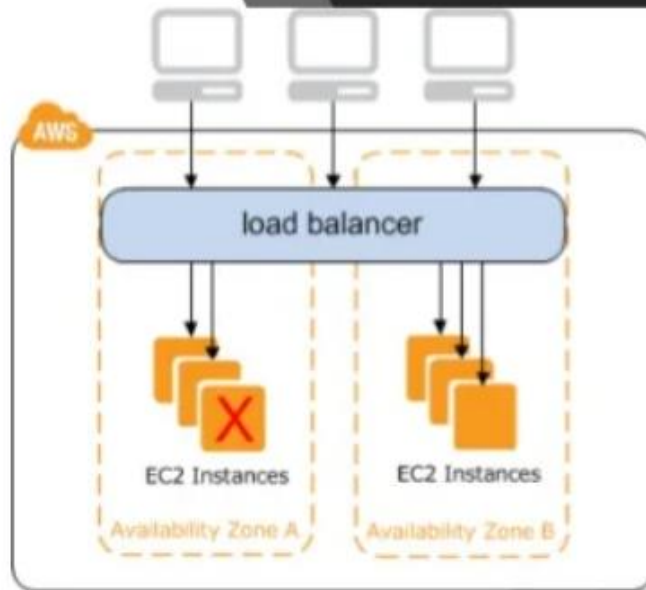
Auto Scaling determines the health status of an instance using one or more of the following:

- Status checks provided by Amazon EC2 (systems status checks and instance status checks)
- Health checks provided by Elastic Load Balancing.

Frequently, an Auto Scaling instance that has just come into service needs to warm up before it can pass the Auto Scaling health check

Auto Scaling waits until the **health check grace period** ends before checking the health status of the instance

Elastic Load Balancer



- A load balancer accepts incoming traffic from clients and routes requests to its registered targets (such as EC2 instances) in one or more Availability Zones
- The load balancer also monitors the health of its registered targets and ensures that it routes traffic only to healthy targets
- You configure your load balancer to accept incoming traffic by specifying one or more listeners
- A listener is a process that checks for connection requests
- It is configured with a protocol and port number for connections from clients to the load balancer and a protocol and port number for connections from the load balancer to the targets

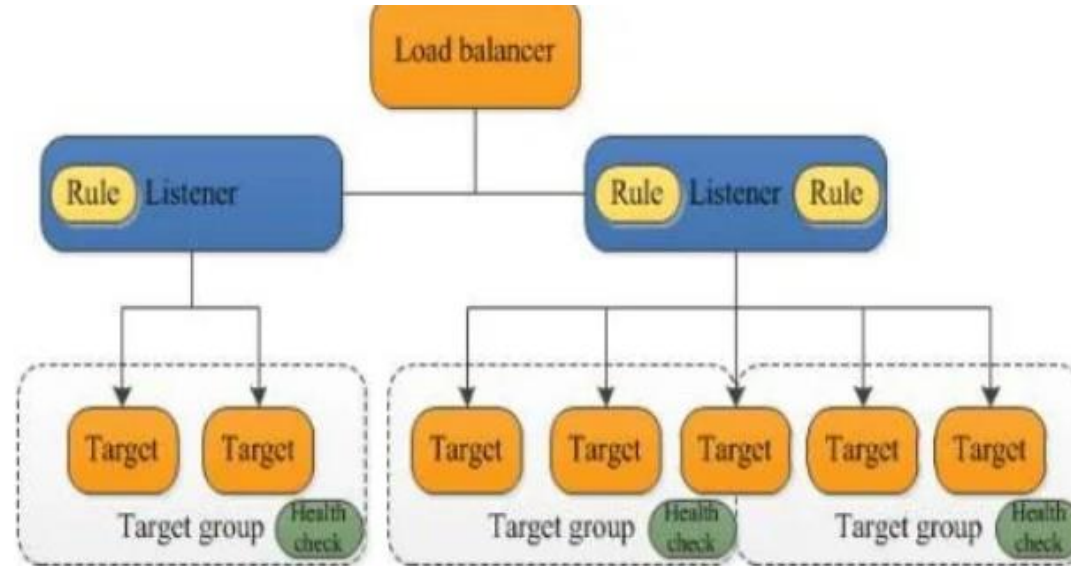
ELB types

Elastic Load Balancing supports three types of load balancers: **Application Load Balancers, Network Load Balancers, and Classic Load Balancers**

With Application Load Balancers and Network Load Balancers, you register targets in target groups, and route traffic to the target groups.

With Classic Load Balancers, you register instances with the load balancer.

Application Load Balancer



- An Application Load Balancer functions at the seventh layer of the Open Systems Interconnection (OSI) model.
- A listener checks for connection requests from clients, using the protocol and port that you configure, and forwards requests to one or more target groups, based on the rules that you define.
- Each rule specifies a target group, condition, and priority. When the condition is met, the traffic is forwarded to the target group

Benefits of Application Load Balancer

- **Support for path-based routing.** You can configure rules for your listener that forward requests based on the URL in the request
- **Support for host-based routing.** You can configure rules for your listener that forward requests based on the host field in the HTTP header.
- **Support for routing requests to multiple applications on a single EC2 instance.** You can register each instance or IP address with the same target group using multiple ports.
- **Support for registering targets by IP address,** including targets outside the VPC for the load balancer.
- **Support for containerized applications**
- **Support for monitoring the health of each service independently,** as health checks are defined at the target group level and many CloudWatch metrics are reported at the target group level
- **Improved load balancer performance**

Benefits of Network Load Balancer

- Ability to handle volatile workloads and scale to millions of requests per second
- Support for static IP addresses for the load balancer. You can also assign one Elastic IP address per subnet enabled for the load balancer
- Support for registering targets by IP address, including targets outside the VPC for the load balancer
- Support for routing requests to multiple applications on a single EC2 instance. You can register each instance or IP address with the same target group using multiple ports
- Support for containerized applications
- Support for monitoring the health of each service independently, as health checks are defined at the target group level and many Amazon CloudWatch metrics are reported at the target group level

Elastic
Beanstalk



Overview

Elastic Beanstalk provides developers and systems administrators an easy, fast way to deploy and manage their applications without having to worry about AWS infrastructure

You simply upload your application, and Elastic Beanstalk automatically handles the details of capacity provisioning, load balancing, scaling, and application health monitoring.

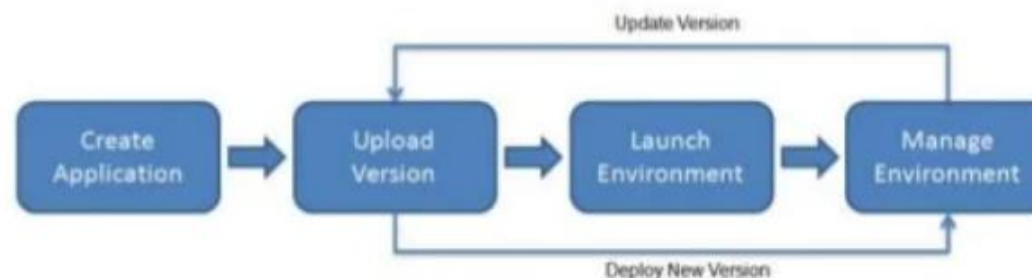
Elastic Beanstalk supports applications developed in Java, PHP, .NET, Node.js, Python, and Ruby, as well as different container types for each language

Elastic Beanstalk automatically launches an environment and creates and configures the AWS resources needed to run your code

After your environment is launched, you can then manage your environment and deploy new application versions

Elastic Beanstalk workflow

To use Elastic Beanstalk, you create an application, upload an application version in the form of an application source bundle (for example, a Java .war file) to Elastic Beanstalk, and then provide some information about the application



AWS Lambda



Lambda

Overview

AWS Lambda is a compute service that lets you run code without provisioning or managing servers

AWS Lambda executes your code only when needed and scales automatically, from a few requests per day to thousands per second

You pay only for the compute time you consume - there is no charge when your code is not running

AWS Lambda runs your code on a high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, code monitoring and logging

All you need to do is supply your code in one of the languages that AWS Lambda supports (currently Node.js, Java, C#, Go and Python)

AWS Lambda Use Case

You can use AWS Lambda to run your code in response to events, such as changes to data in an Amazon S3 bucket or an Amazon DynamoDB table; to run your code in response to HTTP requests using Amazon API Gateway; or invoke your code using API calls made using AWS SDKs.

With these capabilities, you can use Lambda to easily build data processing triggers for AWS services like Amazon S3 and Amazon DynamoDB, process streaming data stored in Kinesis, or create your own back end that operates at AWS scale, performance, and security

This is in exchange for flexibility, which means you cannot log in to compute instances, or customize the operating system or language runtime

Network Services

Virtual Private Cloud

CloudFront

Route53

Direct Connect



VPC

- VPC And Subnets
- Security in VPC
- VPC components
 - Elastic Interfaces
 - Routing Tables
 - Internet Gateways
 - NAT
 - DHCP Options Sets
 - VPC Peering
 - VPC endpoints



VPC

Amazon Virtual Private Cloud (Amazon VPC) enables you to launch AWS resources into a **virtual network** that you've defined.

This virtual network closely resembles a traditional network that you'd operate in your own data center, with the benefits of using the scalable infrastructure of AWS.

Amazon VPC is the **networking layer for Amazon EC2**.

A virtual private cloud (VPC) is a virtual network **dedicated to your AWS account**

You can configure your VPC by modifying its IP address range, create subnets, and configure route tables, network gateways, and security settings

Subnet

A subnet is a range of IP addresses in your VPC.

You can launch AWS resources into a specified subnet

Use a public subnet for resources that must be connected to the internet, and a private subnet for resources that won't be connected to the internet

To protect the AWS resources in each subnet, you can use multiple layers of security, including security groups and network access control lists (ACL)

Default VPC and subnets

Your account comes with a default VPC that has a default subnet in each Availability Zone

A default VPC has the benefits of the advanced features provided by EC2-VPC, and is ready for you to use

If you have a default VPC and don't specify a subnet when you launch an instance, the instance is launched into your default VPC

You can launch instances into your default VPC without needing to know anything about Amazon VPC.

You can create your own VPC, and configure it as you need. This is known as a nondefault VPC

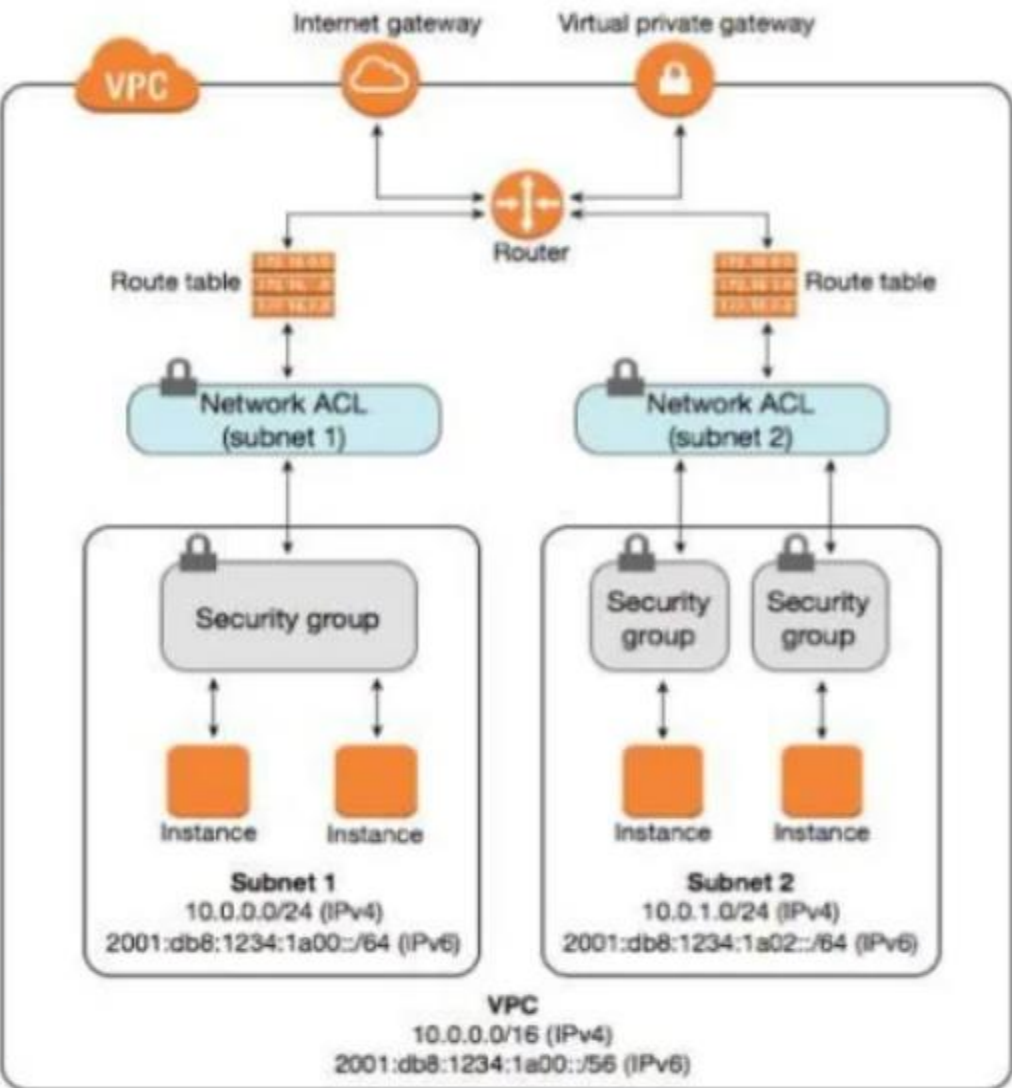
By default, a default subnet is a public subnet, receive both a public IPv4 address and a private IPv4 address

Default VPC Components

When we create a default VPC, AWS do the following to set it up for you:

- Create a VPC with a size /16 IPv4 CIDR block (172.31.0.0/16). This provides up to 65,536 private IPv4 addresses.
- Create a size /20 default subnet in each Availability Zone. This provides up to 4,096 addresses per subnet
- Create an **internet gateway** and connect it to your default VPC
- Create a **main route table** for your default VPC with a rule that sends all IPv4 traffic destined for the internet to the internet gateway
- Create a **default security group** and associate it with your default VPC
- Create a **default network access control list (ACL)** and associate it with your default VPC
- Associate the **default DHCP options set** for your AWS account with your default VPC.

Security in VPC



Security Group vs Network ACL



Security Group

- => Operates at the instance level (first layer of defense)
- => Supports allow rules only
- => Is stateful: Return traffic is automatically allowed, regardless of any rules
- => AWS evaluate all rules before deciding whether to allow traffic
- => Applies to an instance only if someone specifies the security group when launching the instance

•=> Operates at the subnet level (second layer of defense)

=> Supports allow rules and deny rules

=> Is stateless: Return traffic must be explicitly allowed by rules

=> AWS process rules in number order when deciding whether to allow traffic

=> Automatically applies to all instances in the subnets it's associated with

Network ACL

Elastic Network instances

Each instance in your VPC has a default network interface (the **primary network interface**) that is assigned a private IPv4 address

You **cannot detach a primary network interface** from an instance. You can create and attach an additional network interface to any instance in your VPC

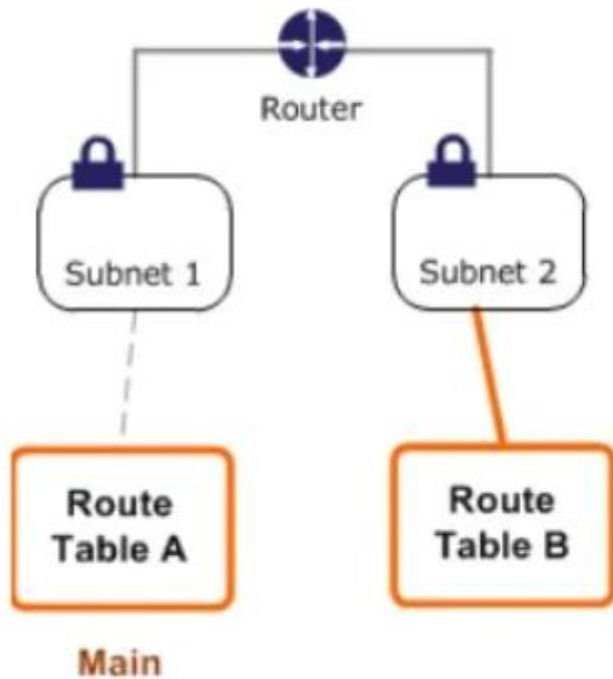
You can create a network interface, attach it to an instance, detach it from an instance, and attach it to another instance

A **network interface's attributes follow it as it is attached or detached** from an instance and reattached to another instance

Attaching multiple network interfaces to an instance is useful when you want to:

- Create a management network.
- Use network and security appliances in your VPC.
- Create dual-homed instances with workloads/roles on distinct subnets
- Create a low-budget, high-availability solution.

Routing Table



- A route table contains a set of rules, called routes, that are used to determine where network traffic is directed
- Your VPC has an **implicit router**.
- Your VPC automatically comes with a main route table that you can modify.
- You can create additional custom route tables for your VPC
- Each subnet in your VPC must be associated with a route table; the table controls the routing for the subnet
- A subnet can only be associated with one route table at a time, but you can associate multiple subnets with the same route table
- If you don't explicitly associate a subnet with a particular route table, the subnet is implicitly associated with the main route table.
- You cannot delete the main route table, but you can replace the main route table with a custom table that you've created
- Every route table contains a local route for communication within the VPC over IPv4.

Internet Gateway

- An Internet gateway is a **horizontally scaled, redundant, and highly available** VPC component that allows communication between instances in your VPC and the Internet
- It therefore imposes no availability risks or bandwidth constraints on your network traffic
- An Internet gateway **supports IPv4 and IPv6 traffic**.
- To enable access to or from the Internet for instances in a VPC subnet, you must do the following:
 - Attach an Internet gateway to your VPC.
 - Ensure that your subnet's route table points to the Internet gateway.
 - Ensure that instances in your subnet have a globally unique IP address (public IPv4 address, Elastic IP address, or IPv6 address)
 - Ensure that your network access control and security group rules allow the relevant traffic to flow to and from your instance.



NAT

- You can use a NAT device to enable instances in a private subnet to connect to the Internet or other AWS services, but prevent the Internet from initiating connections with the instances.
- A NAT device forwards traffic from the instances in the private subnet to the Internet or other AWS services, and then sends the response back to the instances
- When traffic goes to the Internet, the source IPv4 address is replaced with the NAT device's address and similarly, when the response traffic goes to those instances, the NAT device translates the address back to those instances' private IPv4 addresses.
- AWS offers two kinds of NAT devices—a **NAT gateway** or a **NAT instance**.
- AWS recommends NAT gateways, as they provide better availability and bandwidth over NAT instances
- The NAT Gateway service is also a managed service that does not require your administration efforts
- A NAT instance is launched from a NAT AMI.



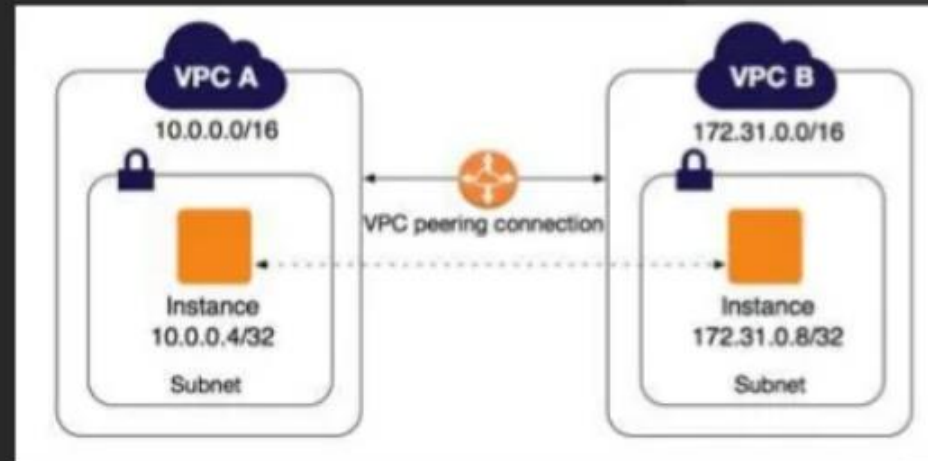
DHCP Option sets

- The DHCP options provides a standard for passing configuration information to hosts on a TCP/IP network such as domain name, domain name server, NTP servers.
- DHCP options sets are associated with your AWS account so that you can use them across all of your virtual private clouds (VPC)
- After you create a set of DHCP options, you can't modify them
- If you want your VPC to use a different set of DHCP options, you must create a new set and associate them with your VPC
- You can also set up your VPC to use no DHCP options at all.
- You can have multiple sets of DHCP options, but you can associate only one set of DHCP options with a VPC at a time
- After you associate a new set of DHCP options with a VPC, any existing instances and all new instances use these options within few hours.

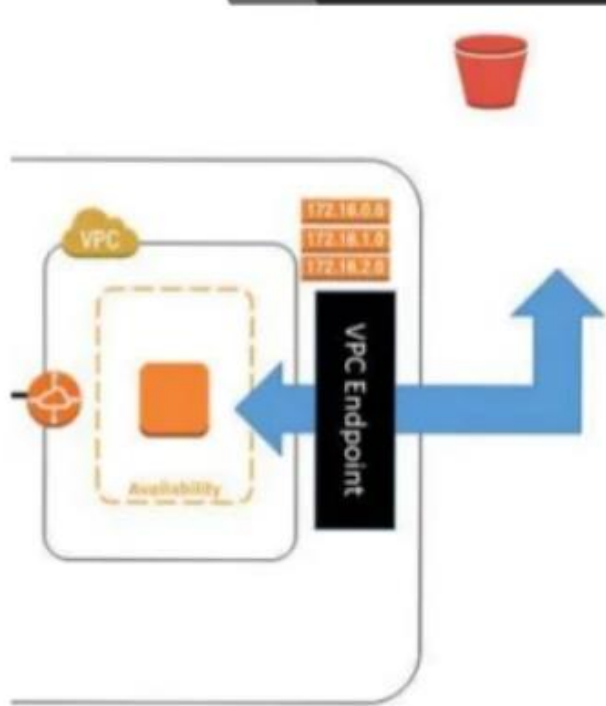


VPC Peering

- A VPC peering connection is a networking **connection between two VPCs** that enables you to route traffic between them privately
- Instances in either VPC can communicate with each other as if they are within the same network.
- You can create a VPC peering connection between your own VPCs, with a VPC in another AWS account, or with a VPC in a different AWS Region
- There **should not be any overlapping of IP addresses** as a pre-requisite for setting up the VPC peering



VPC Endpoints



- A VPC endpoint enables you to privately connect your VPC to supported AWS services and VPC endpoint services powered by PrivateLink without requiring an internet gateway
- Instances in your VPC do not require public IP addresses to communicate with resources in the service.
- Traffic between your VPC and the other service does not leave the Amazon network
- Endpoints are horizontally scaled, redundant, and highly available VPC components without imposing availability risks or bandwidth constraints on your network traffic

There are two types of VPC endpoints based on the supported target services:

1. **Interface endpoint interfaces** : An elastic network interface with a private IP address that serves as an entry point for traffic destined to a supported service
2. **Gateway endpoint interfaces** : A gateway that is a target for a specified route in your route table, used for traffic destined to a supported AWS service.



Overview

CDN/CloudFront can be used in every use case where the web services or media files are provided to end users and the end users are spread across geographies

Amazon CloudFront is a web service that speeds up distribution of your static and dynamic web content, such as .html, .css, .js, and image files, to your users

CloudFront delivers your content through a worldwide network of data centers called edge locations

Benefits of CDN

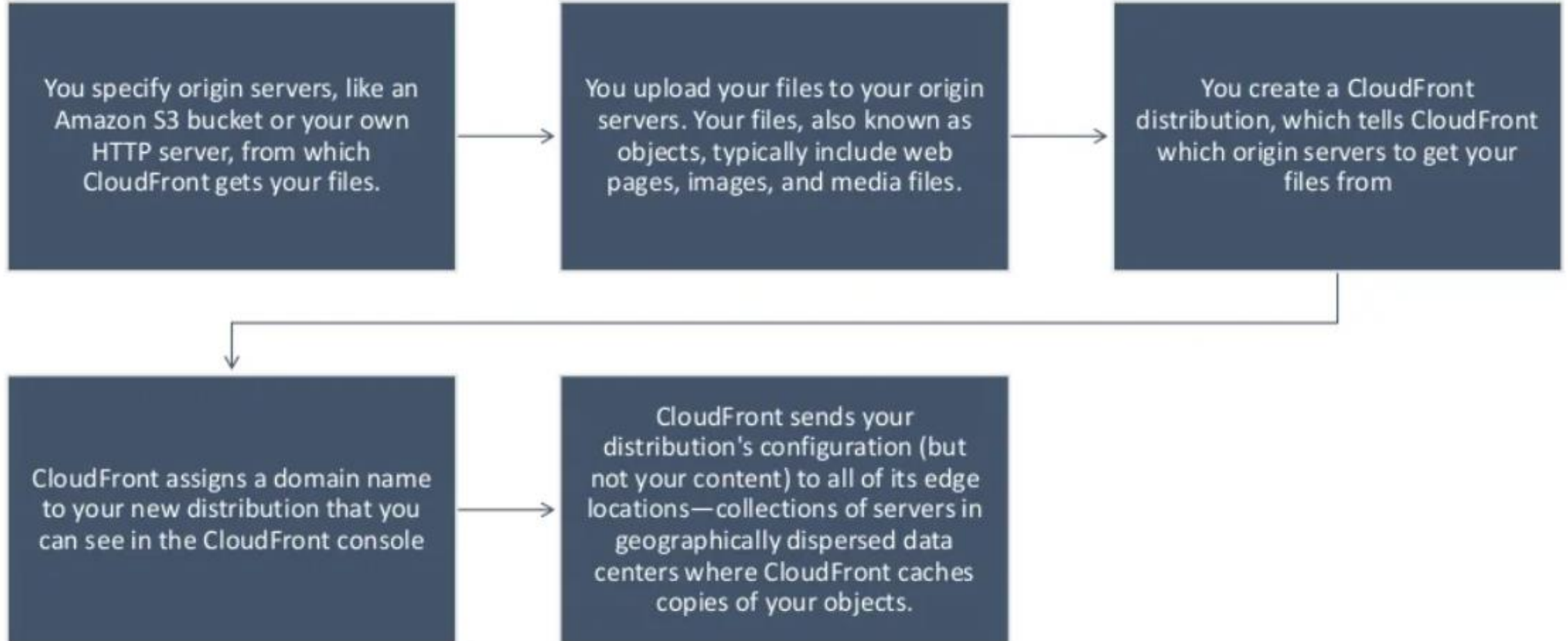
Better customer experience with faster page load

Reduced load on origin (source) servers

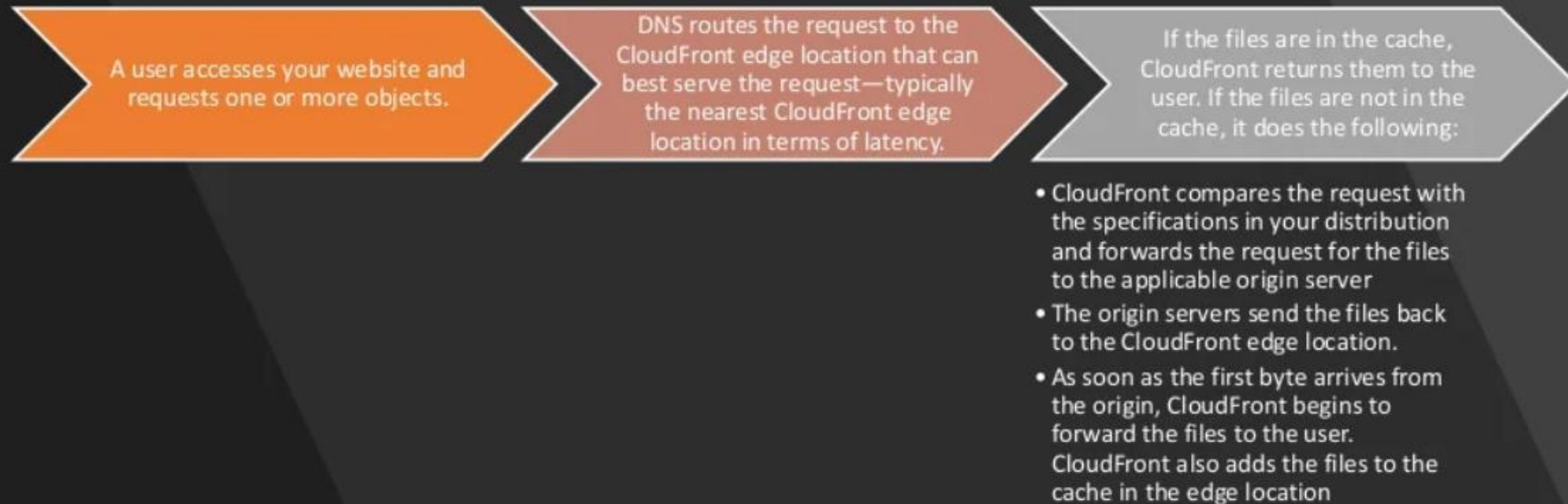
Reliable and highly available even when the origin server is down

Protection from DDOS attacks

Configuring CloudFront



CloudFront Content Delivery



Route53



Overview

Route 53 performs three main functions:

- **Register** domain names
- **Route internet traffic** to the resources for your domain
- **Check the health** of your resources

A hosted zone is a container for records, and records contain information about how you want to route traffic for a specific domain



There are two types of hosted zones supported by Route53:

Public hosted zones contain records that specify how you want to route traffic on the internet

Private hosted zones contain records that specify how you want to route traffic in an Amazon VPC.

Hosted Zone

Routing Policies

When you create a record, you choose a routing policy, which determines how Amazon Route 53 responds to queries:

- **Simple** Routing Policy
- **Failover** routing policy
- **Geolocation** routing policy
- **Geoproximity** routing policy
- **Latency** routing policy
- **Multivalue** answer routing policy
- **Weighted** routing policy

AWS Direct Connect



Overview

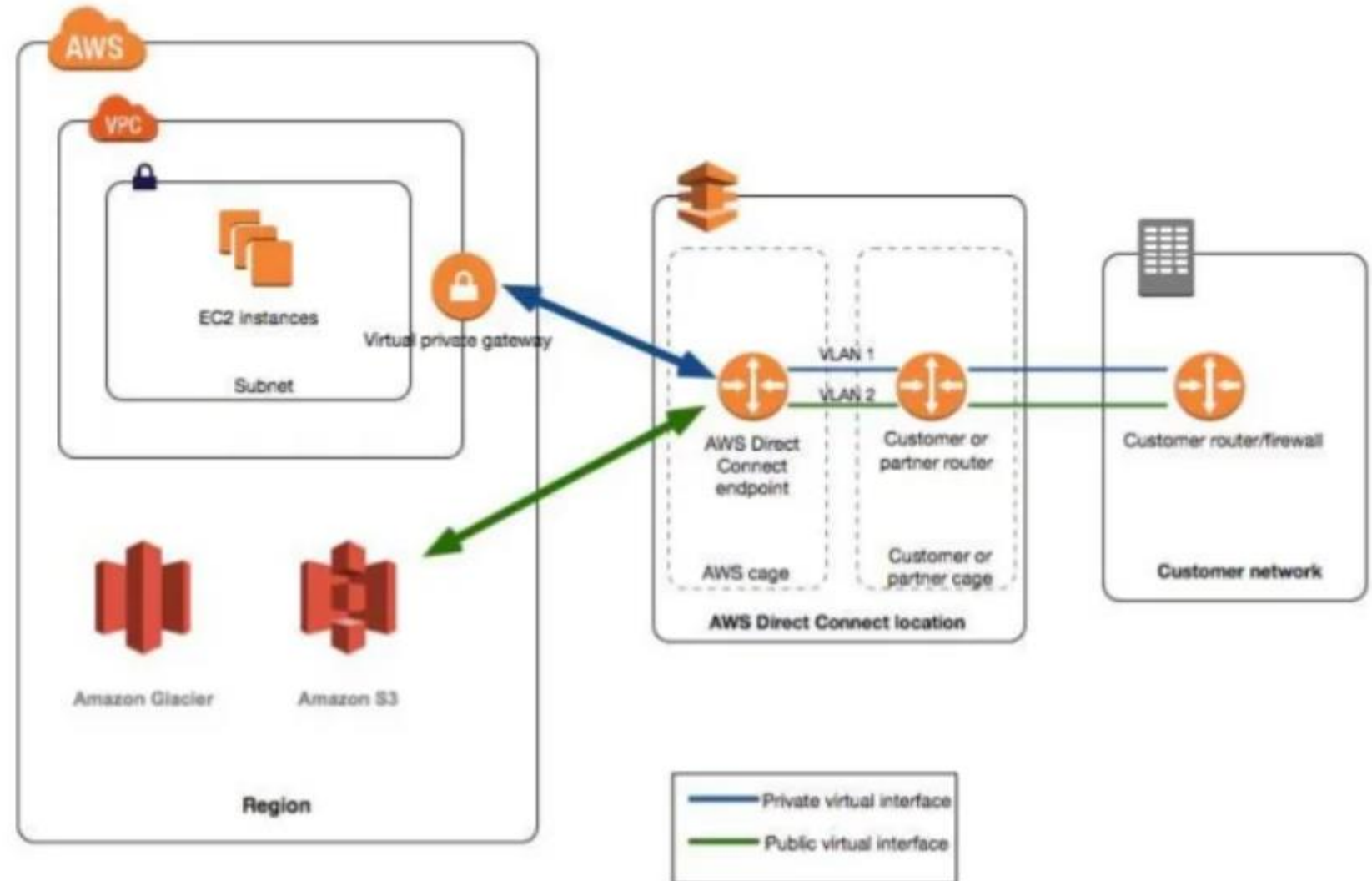
AWS Direct Connect makes it easy to establish a dedicated network connection from your premises to AWS

AWS Direct Connect links your internal network to an AWS Direct Connect location over a standard 1-gigabit or 10-gigabit Ethernet fiber-optic cable

Using industry standard 802.1q VLANs, this dedicated connection can be partitioned into multiple virtual interfaces

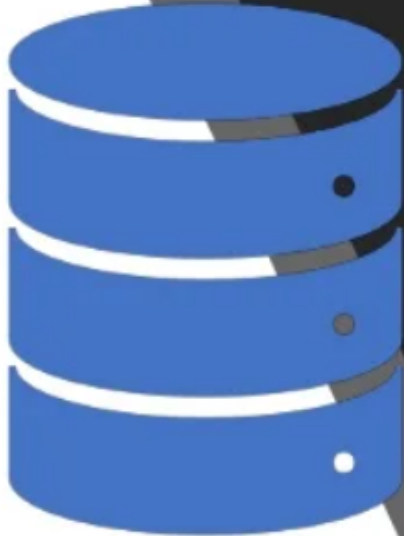
A public virtual interface enables access to public-facing services, such as Amazon S3. A private virtual interface enables access to your VPC

AWS Direct Connect



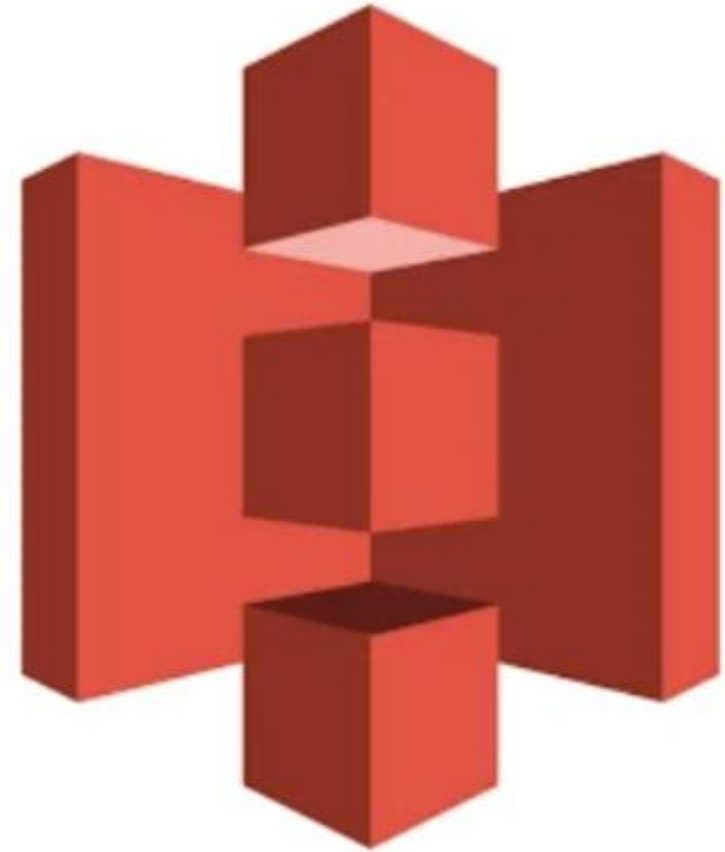
Storage Services

- S3
- EBS
- Storage Gateway



S3

- S3 features
- Key Concepts
- Storage classes
- Versioning
- Managing access



| S3

Amazon Simple Storage Service is storage for the Internet.

It is designed to make web-scale computing easier for developers.

S3 is designed to provide 99.999999999% durability and 99.99% availability of objects over a given year

S3 features

Storage Classes

Bucket Policies & Access Control Lists

Versioning

Data encryption

Lifecycle Management

Cross Region Replication

S3 transfer Acceleration

Requester pays

S3 analytics and Inventory

Key Concepts : Objects

- Objects are the fundamental entities stored in Amazon S3
- An object consists of the following:
 - **Key** – The name that you assign to an object. You use the object key to retrieve the object.
 - **Version ID** – Within a bucket, a key and version ID uniquely identify an object. The version ID is a string that Amazon S3 generates when you add an object to a bucket.
 - **Value** – The content that you are storing. An object value can be any sequence of bytes. Objects can range in size from zero to 5 TB
 - **Metadata** – A set of name-value pairs with which you can store information regarding the object. You can assign metadata, referred to as user-defined metadata
 - **Access Control Information** – You can control access to the objects you store in Amazon S3

Key Concepts : Buckets



- A bucket is a *container for objects* stored in Amazon S3.
- Every object is contained in a bucket.
- Amazon S3 bucket names are *globally unique*, regardless of the AWS Region in which you create the bucket.
- A bucket is owned by the AWS account that created it.
- Bucket ownership is not transferable;
- There is *no limit to the number of objects* that can be stored in a bucket and no difference in performance whether you use many buckets or just a few
- You cannot create a bucket within another bucket.

Key Concepts : Object key

- Every object in Amazon S3 can be uniquely addressed through the combination of the *web service endpoint, bucket name, key, and optionally, a version*.
- For example, in the URL `http://doc.s3.amazonaws.com/2006-03-01/AmazonS3.wsdl`, "doc" is the name of the bucket and "2006-03-01/AmazonS3.wsdl" is the key.

Storage Class

Each object in Amazon S3 has a storage class associated with it.

Amazon S3 offers the following storage classes for the objects that you store

- STANDARD
- STANDARD_IA
- GLACIER

This storage class is ideal for performance-sensitive use cases and frequently accessed data.

STANDARD is the default storage class; if you don't specify storage class at the time that you upload an object, Amazon S3 assumes the STANDARD storage class.

Designed for Durability : 99.999999999%

Designed for Availability : 99.99%

Standard class

This storage class (IA, for infrequent access) is **optimized for long-lived and less frequently accessed data**

for example backups and older data where frequency of access has diminished, but the use case still demands high performance.

There is a **retrieval fee** associated with STANDARD_IA objects which makes it most suitable for infrequently accessed data.

The STANDARD_IA storage class is suitable for larger objects greater than 128 Kilobytes that you want to keep for at least 30 days

Designed for durability : 99.999999999%

Designed for Availability : 99.9%

Standard_IA class

Glacier

- The GLACIER storage class is **suitable for archiving data** where data access is infrequent
- Archived objects are **not available for real-time access**. You must first restore the objects before you can access them.
- You cannot specify GLACIER as the storage class at the time that you create an object.
- You create GLACIER objects by first uploading objects using STANDARD, RRS, or STANDARD_IA as the storage class. Then, you transition these objects to the GLACIER storage class using lifecycle management.
- You must first restore the GLACIER objects before you can access them
- Designed for durability : 99.999999999%
- Designed for Availability : 99.99%

Reduced_Redundance Storage class

RRS storage class is **designed for noncritical, reproducible data stored** at lower levels of redundancy than the STANDARD storage class.

if you store 10,000 objects using the RRS option, you can, on average, expect to incur an annual loss of a single object per year (0.01% of 10,000 objects)

Amazon S3 can send an event notification to alert a user or start a workflow when it detects that an RRS object is lost

Designed for durability : 99.99%

Designed for Availability : 99.99%

Lifecycle Management

- Using lifecycle configuration rules, you can direct **S3 to tier down the storage classes, archive, or delete the objects** during their lifecycle.
- The configuration is a set of one or more rules, where each rule defines an action for Amazon S3 to apply to a group of objects
- These actions can be classified as follows:

Transition

- In which you define when objects transition to another storage class.

Expiration

- In which you specify when the objects expire. Then Amazon S3 deletes the expired objects on your behalf.

When Should I Use Lifecycle Configuration?

If you are uploading periodic logs to your bucket, your application might need these logs for a week or a month after creation, and after that you might want to delete them.

Some documents are frequently accessed for a limited period of time. After that, these documents are less frequently accessed. Over time, you might not need real-time access to these objects, but your organization or regulations might require you to archive them for a longer period

You might also upload some types of data to Amazon S3 primarily for archival purposes, for example digital media archives, financial and healthcare records etc

Versioning

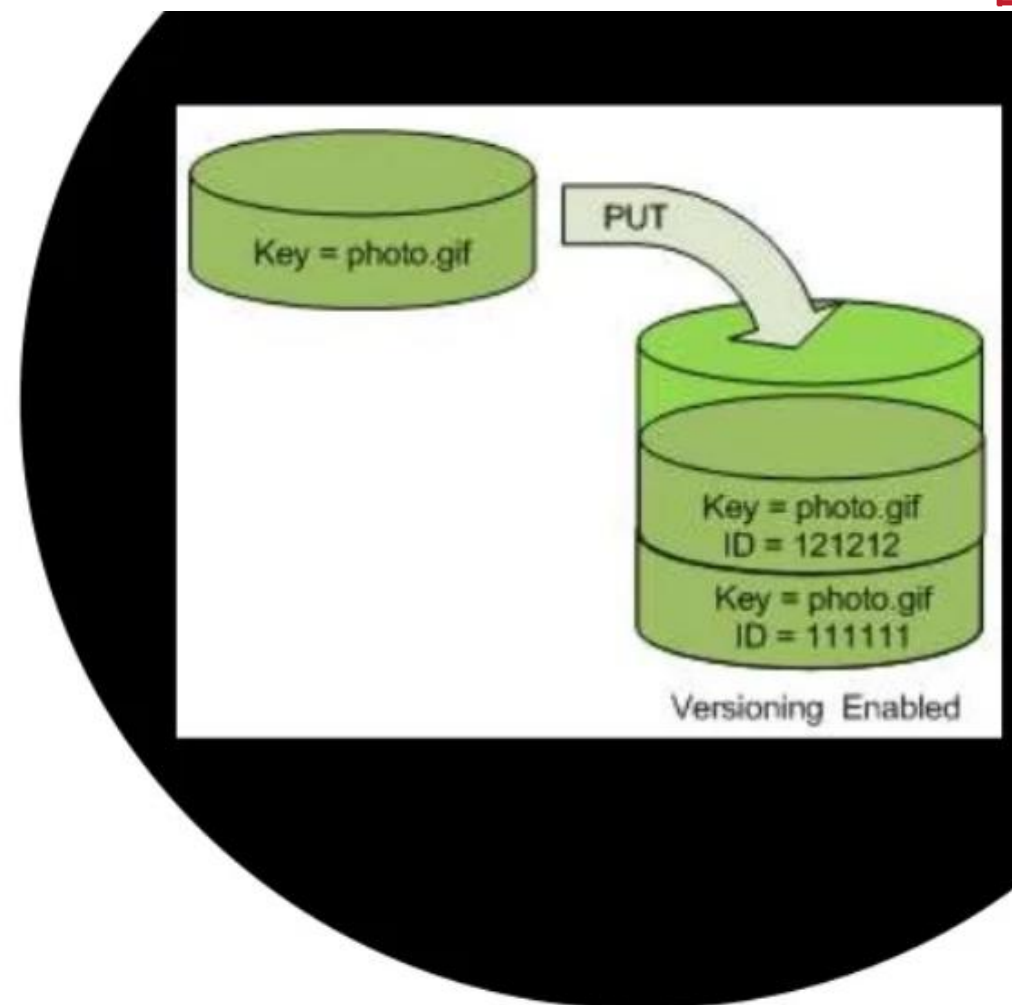
- Versioning enables you to keep multiple versions of an object in one bucket.
- Once versioning is enabled, it **can't be disabled but can be suspended**
- Enabling and suspending versioning is done at the bucket level
- You might want to enable versioning to protect yourself from unintended overwrites and deletions or to archive objects so that you can retrieve previous versions of them
- You must explicitly enable versioning on your bucket. **By default, versioning is disabled**
- Regardless of whether you have enabled versioning, **each object in your bucket has a version ID**

Versioning (contd..)

- If you have not enabled versioning, then Amazon S3 sets the version ID value to null.
- If you have enabled versioning, Amazon S3 assigns a unique version ID value for the object
- An example version ID is 3/L4kqtJlcpXroDTDmJ+rmSpXd3dlbrHY+MTRCxf3vjVBH40Nr8X8gdRQBpUMLUo. Only Amazon S3 generates version IDs. They cannot be edited.
- When you enable versioning on a bucket, existing objects, if any, in the bucket are unchanged: the version IDs (null), contents, and permissions remain the same

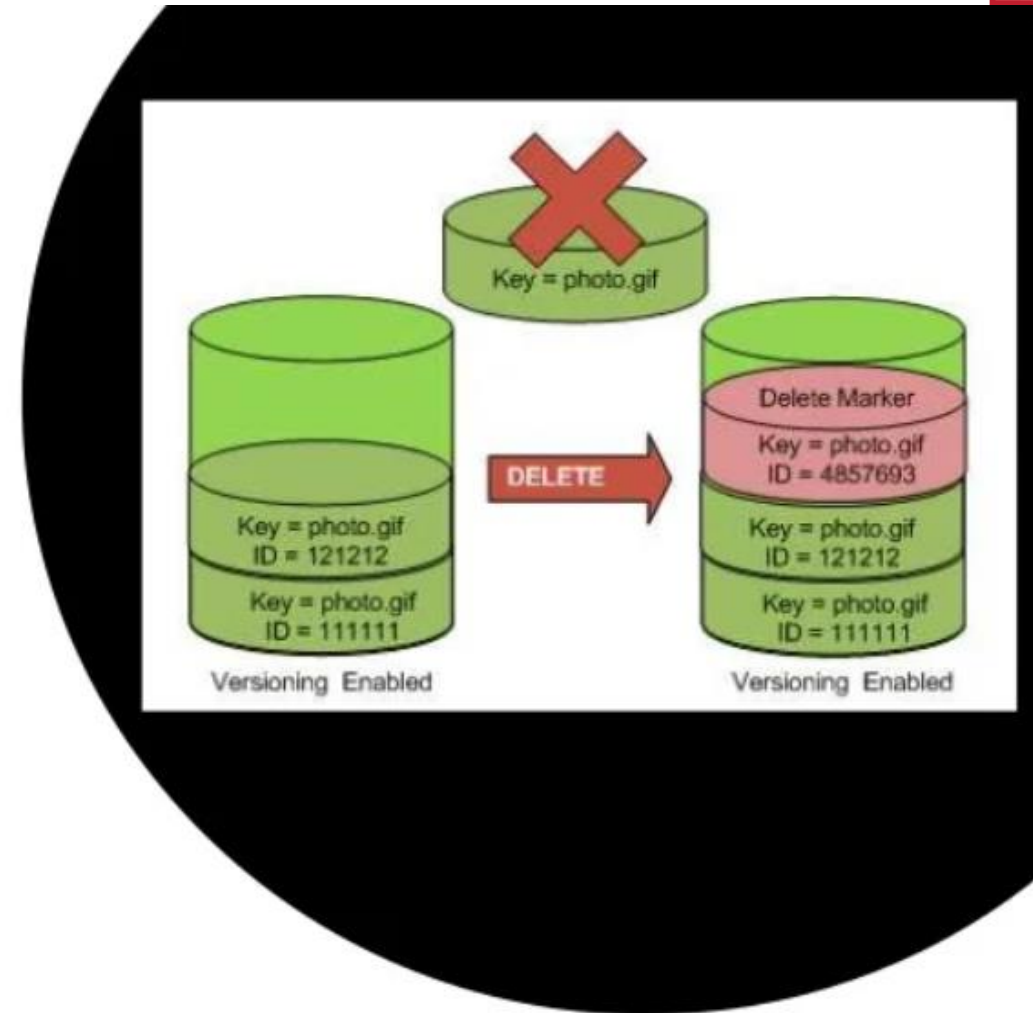
Versioning : PUT Operation

- When you PUT an object in a versioning-enabled bucket, the noncurrent version is not overwritten.
- The following figure shows that when a new version of photo.gif is PUT into a bucket that already contains an object with the same name, S3 generates a new version ID (121212), and adds the newer version to the bucket.

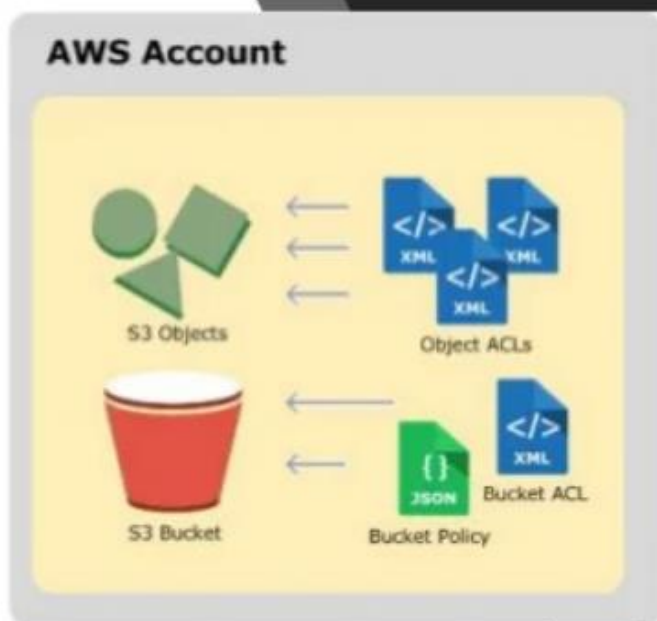


Versioning : DELETE Operation

- When you DELETE an object, all versions remain in the bucket and Amazon S3 inserts a **delete marker**.
- The delete marker becomes the current version of the object. By default, GET requests retrieve the most recently stored version. Performing a simple GET Object request when the current version is a delete marker returns a 404 Not Found error
- You can, however, GET a noncurrent version of an object by specifying its version ID
- You can permanently delete an object by specifying the version you want to delete.



Managing access



- By default, all Amazon S3 resources—buckets, objects, and related subresources are **private** : only the resource owner, an AWS account that created it, can access the resource.
- The resource owner can optionally grant access permissions to others by writing an **access policy**
- Amazon S3 offers access policy options broadly categorized as **resource-based policies and user policies**.
 - Access policies you attach to your resources are referred to as resource-based policies. For example, bucket policies and access control lists (ACLs) are resource-based policies.
 - You can also attach access policies to users in your account. These are called user policies

Resource Owner

AWS Account (Resource Owner)

IAM Users



Admin User

Users

Amazon S3 Resources



S3 Objects

S3 Bucket

- The AWS account that you use to create buckets and objects owns those resources.
- If you create an IAM user in your AWS account, your AWS account is the parent owner. If the IAM user uploads an object, the parent account, to which the user belongs, owns the object.
- A bucket owner can grant cross-account permissions to another AWS account (or users in another account) to upload objects
- In this case, the AWS account that uploads objects owns those objects. The bucket owner does not have permissions on the objects that other accounts own, with the following exceptions:
 - The bucket owner pays the bills. The bucket owner can deny access to any objects, or delete any objects in the bucket, regardless of who owns them
 - The bucket owner can archive any objects or restore archived objects regardless of who owns them

When to Use an ACL-based Access Policy

An object ACL is the only way to manage access to objects not owned by the bucket owner

Permissions vary by object and you need to manage permissions at the object level

Object ACLs control only object-level permissions

EBS

An Amazon EBS volume is a durable, block-level storage device that you can attach to a single EC2 instance.

EBS volumes are particularly well-suited for use as the primary storage for file systems, databases, or for any applications that require fine granular updates and access to raw, unformatted, block-level storage

EBS volumes are created in a specific Availability Zone, and can then be attached to any instances in that same Availability Zone.

While creating an EBS volume , AWS does industry standard disk wiping

Benefits of EBS Volume

Data Availability: When you create an EBS volume in an Availability Zone, it is automatically replicated within that zone to prevent data loss due to failure of any single hardware component

Data persistence: An EBS volume is off-instance storage that can persist independently from the life of an instance

Data encryption: For simplified data encryption, you can create encrypted EBS volumes with the Amazon EBS encryption feature.

Snapshots: Amazon EBS provides the ability to create snapshots (backups) of any EBS volume and write a copy of the data in the volume to Amazon S3, where it is stored redundantly in multiple Availability Zones.

Flexibility: EBS volumes support live configuration changes while in production. You can modify volume type, volume size, and IOPS capacity without service interruptions.

EBS Volume Types

Amazon EBS provides the following volume types, which differ in performance characteristics and price.

The volumes types fall into two categories:

- **SSD-backed volumes** optimized for transactional workloads involving frequent read/write operations with small I/O size, where the dominant performance attribute is IOPS (gp2, io1)
- **HDD-backed volumes** optimized for large streaming workloads where throughput (measured in MiB/s) is a better performance measure than IOPS (St1, Sc1)

General purpose SSD volumes (gp2)

- **Description** : General purpose SSD volume that balances price and performance for a wide variety of workloads
- **Use Cases**: Recommended for most workloads , System boot volumes , Low-latency interactive apps , Development and test environments
- **API Name** : Gp2
- **Volume Size** : 1 GiB - 16 TiB
- **Max IOPS** : 10,000
- **Max throughput** : 160 MiB/s
- **Max IOPS/ Instance** : 80,000
- **Minimum IOPS** : 100
- Between a minimum of 100 IOPS (at 33.33 GiB and below) and a maximum of 10,000 IOPS (at 3,334 GiB and above), baseline performance scales linearly at 3 IOPS per GiB of volume size

Gp2 volumes IO credits and Burst performance

- The performance of gp2 volumes is tied to volume size
- Volume Size determines the baseline performance level of the volume and how quickly it accumulates I/O credits
- larger volumes have higher baseline performance levels and accumulate I/O credits faster
- I/O credits represent the available bandwidth that your gp2 volume can use to burst large amounts of I/O when more than the baseline performance is needed
- Each volume receives an initial I/O credit balance of 5.4 million I/O credits, which is enough to sustain the maximum burst performance of 3,000 IOPS for 30 minutes
- This initial credit balance is designed to provide a fast initial boot cycle for boot volumes and to provide a good bootstrapping experience for other applications
- If you notice that your volume performance is frequently limited to the baseline level , you should consider using a larger gp2 volume or switching to an io1 volume

Provisioned IOPS SSD volumes (io1)

- **Description** : Highest-performance SSD volume for mission-critical low-latency or high-throughput workloads
- **Use case** : Critical business applications that require sustained IOPS performance , Large database workloads
- **API Name** : io1
- **Volume Size** : 4 GiB - 16 TiB
- **MAX IOPS** : 32,000
- **MAX Throughput** : 500 MiB/s
- **MAX IOPS per instance** : 80000

Throughput Optimized HDD Volumes (st1)

- **Description** : Low cost HDD volume designed for frequently accessed, throughput-intensive workloads
- **Use Cases** : Streaming workloads requiring consistent, fast throughput at a low price , Big Data , Data warehouse , log data , cant be a boot volume
- **API name** : st1
- **Volume Size** : 500 GiB - 16 TiB
- **Max. Throughput/Volume** : 500 MiB/s
- **Throughput Credits and Burst Performance** :
 - Like gp2, st1 uses a burst-bucket model for performance.
 - Volume size determines the baseline throughput of your volume, which is the rate at which the volume accumulates throughput credits
 - For a 1-TiB st1 volume, burst throughput is limited to 250 MiB/s, the bucket fills with credits at 40 MiB/s, and it can hold up to 1 TiB-worth of credits.

Cold HDD volumes (sc1)

- **Description:** Lowest cost HDD volume designed for less frequently accessed workloads
- **Use Cases:** Throughput-oriented storage for large volumes of data that is infrequently accessed , Scenarios where the lowest storage cost is important, Can't be a boot volume
- **Api Name :** sc1
- **Volume Size :** 500 GiB - 16 TiB
- **Max. Throughput/Volume :** 250 MiB/s
- **Throughput Credits and Burst Performance:**
 - Like gp2, sc1 uses a burst-bucket model for performance.
 - Volume size determines the baseline throughput of your volume, which is the rate at which the volume accumulates throughput credits.
 - For a 1-TiB sc1 volume, burst throughput is limited to 80 MiB/s, the bucket fills with credits at 12 MiB/s, and it can hold up to 1 TiB-worth of credits.

EBS Snapshots

- You can back up the data on your Amazon EBS volumes to Amazon S3 by taking point-in-time snapshots.
- Snapshots are incremental backups, which means that only the blocks on the device that have changed after your most recent snapshot are saved.
- This minimizes the time required to create the snapshot and saves on storage costs by not duplicating data
- When you delete a snapshot, only the data unique to that snapshot is removed.
- Each snapshot contains all of the information needed to restore your data (from the moment when the snapshot was taken) to a new EBS volume
- When you create an EBS volume based on a snapshot, the new volume begins as an exact replica of the original volume that was used to create the snapshot.
- You can share a snapshot across AWS accounts by modifying its access permissions
- You can also copy snapshots across regions, making it possible to use multiple regions for geographical expansion, data center migration, and disaster recovery

Amazon EBS Optimized instances

- An Amazon EBS-optimized instance uses an optimized configuration stack and provides additional, dedicated capacity for Amazon EBS I/O
- EBS-optimized instances deliver dedicated bandwidth to Amazon EBS, with options between 425 Mbps and 14,000 Mbps, depending on the instance type you use
- The instance types that are EBS-optimized by default, there is no need to enable EBS optimization and no effect if you disable EBS optimization
- For instances that are not EBS-optimized by default, you can enable EBS optimization
- When you enable EBS optimization for an instance that is not EBS-optimized by default, you pay an additional low, hourly fee for the dedicated capacity.
- Example of instances which are EBS -optimized by default : C4, C5, d3, f1, g3, h1, i3, m4 m5, r4, X1 , P2, P3

Amazon EBS Encryption

When you create an encrypted EBS volume and attach it to a supported instance type, the following types of data are encrypted:

- Data at rest inside the volume
- All data moving between the volume and the instance
- All snapshots created from the volume
- All volumes created from those snapshots

Encryption operations occur on the servers that host EC2 instances, ensuring the security of both data-at-rest and data-in-transit between an instance and its attached EBS storage

Snapshots of encrypted volumes are automatically encrypted.

Volumes that are created from encrypted snapshots are automatically encrypted.

By using the AWS Storage Gateway software appliance, you can connect your existing on-premises application infrastructure with scalable, cost-effective AWS cloud storage that provides data security features

AWS Storage Gateway offers file-based, volume-based, and tape-based storage solutions

Gateway is a software appliance installed as VM at your On-premise Virtualization infrastructure (ESX/ HyperV) or an EC2 at the AWS infrastructure

To prepare for upload to Amazon S3, your gateway also stores incoming data in a staging area, referred to as an upload buffer

Your gateway uploads this buffer data over an encrypted Secure Sockets Layer (SSL) connection to AWS, where it is stored encrypted in Amazon S3

A graphic at the bottom of the slide featuring two overlapping trapezoidal shapes. The left shape is light gray and contains the text 'Storage Gateway'. The right shape is a darker gray and is empty.

Storage Gateway

File Gateway

The gateway provides access to objects in S3 as files on an NFS mount point

Objects are encrypted with server-side encryption with Amazon S3–managed encryption keys (SSE-S3).

All data transfer is done through HTTPS

The service optimizes data transfer between the gateway and AWS using multipart parallel uploads or byte-range downloads

A local cache is maintained to provide low latency access to the recently accessed data and reduce data egress charges

Volume gateway

A volume gateway provides cloud-backed storage volumes that you can mount as Internet Small Computer System Interface (iSCSI) devices from your on-premises application servers.

You can create storage volumes and mount them as iSCSI devices from your on-premises application servers

The gateway supports the following volume configurations:

Cached volumes

Stored Volumes

Cached volumes

- By using cached volumes, you can use Amazon S3 as your primary data storage, while retaining frequently accessed data locally in your storage gateway.
- Cached volumes minimize the need to scale your on-premises storage infrastructure, while still providing your applications with low-latency access to their frequently accessed data.
- Cached volumes can range from 1 GiB to 32 TiB in size and must be rounded to the nearest GiB.
- Each gateway configured for cached volumes can support up to 32 volumes for a total maximum storage volume of 1,024 TiB (1 PiB).
- Generally, you should allocate at least 20 percent of your existing file store size as cache storage.
- You can take incremental backups, called snapshots, of your storage volumes in Amazon S3.
- All gateway data and snapshot data for cached volumes is stored in Amazon S3 and encrypted at rest using server-side encryption (SSE).
- However, you can't access this data with the Amazon S3 API or other tools such as the Amazon S3 Management Console.

Stored Volumes

By using stored volumes, you can store your primary data locally, while asynchronously backing up that data to AWS S3 as EBS snapshots.

This configuration provides durable and inexpensive offsite backups that you can recover to your local data center or Amazon EC2

Stored volumes can range from 1 GiB to 16 TiB in size and must be rounded to the nearest GiB

Each gateway configured for stored volumes can support up to 32 volumes and a total volume storage of 512 TiB (0.5 PiB).

Tape Gateway

With a tape gateway, you can cost-effectively and durably archive backup data in Amazon Glacier.

A tape gateway provides a virtual tape infrastructure that scales seamlessly with your business needs and eliminates the operational burden of provisioning, scaling, and maintaining a physical tape infrastructure.

With its virtual tape library (VTL) interface, you use your existing tape-based backup infrastructure to store data on virtual tape cartridges that you create on your tape gateway

Database Services

RDS

DynamoDB

Redshift

Elasticache

RDS

- RDS features
- DB Instances
- High Availability (Multi-AZ)
- Read Replicas
- Parameter Groups
- Backup & Restore
- Monitoring
- RDS Security



RDS

Amazon Relational Database Service (Amazon RDS) is a web service that makes it easier to set up, operate, and scale a relational database in the cloud.

It provides cost-efficient, resizable capacity for an industry-standard relational database and manages common database administration tasks

RDS features

- When you buy a server, you get CPU, memory, storage, and IOPS, all bundled together. With Amazon RDS, these are split apart so that you can **scale them independently**
- Amazon **RDS manages backups, software patching, automatic failure detection, and recovery.**
- To deliver a managed service experience, Amazon **RDS doesn't provide shell access to DB instances**
- You can have **automated backups** performed when you need them, or manually create your own **backup snapshot**.
- You can get **high availability** with a primary instance and a synchronous secondary instance that you can fail over to when problems occur
- You can also use MySQL, MariaDB, or PostgreSQL **Read Replicas** to increase read scaling.
- In addition to the security in your database package, you can help control who can access your RDS databases by using AWS Identity and Access Management (IAM)
- Supports the popular engines : **MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server, and the new, MySQL-compatible Amazon Aurora DB engine**

DB instances

- The basic building block of Amazon RDS is the **DB instance**
- A DB instance can contain multiple user-created databases, and you can access it by using the same tools and applications that you use with a stand-alone database instance
- Each DB instance runs a DB engine. Amazon RDS currently supports the MySQL, MariaDB, PostgreSQL, Oracle, and Microsoft SQL Server DB engines
- When creating a DB instance, some database engines require that a **database name** be specified.
- Amazon RDS creates a **master user account** for your DB instance as part of the creation process

DB instance Class

- The DB instance class determines the computation and memory capacity of an Amazon RDS DB instance
- Amazon RDS supports three types of instance classes: Standard, Memory Optimized, and Burstable Performance.
- DB instance storage comes in three types: Magnetic, General Purpose (SSD), and Provisioned IOPS (PIOPS).

Standard DB instance classes : db.m4,db.m3, db.m1

Memory Optimized DB instance classes: db.r4, db.r3,

Burstable Performance DB instance class: db.t2

High Availability (Multi-AZ)

- Amazon RDS provides high availability and failover support for DB instances using **Multi-AZ deployments**
- In a Multi-AZ deployment, Amazon RDS automatically provisions and maintains a **synchronous standby replica** in a different Availability Zone
- The high-availability feature is not a scaling solution for read-only scenarios; you **cannot use a standby replica to serve read traffic**.
- DB instances using Multi-AZ deployments **may have increased write and commit latency** compared to a Single-AZ deployment



Failover Process for Amazon RDS

- In the event of a planned or unplanned outage of your DB instance, RDS **automatically switches to a standby replica** in another Availability Zone
- Failover times are typically **60-120 seconds**. However, large transactions or a lengthy recovery process can increase failover time
- The failover mechanism **automatically changes the DNS record** of the DB instance to point to the standby DB instance
- As a result, you need to **re-establish any existing connections** to your DB instance.

Failover Cases

- The primary DB instance switches over automatically to the standby replica if any of the following conditions occur:
 - An Availability Zone outage
 - The primary DB instance fails
 - The DB instance's server type is changed
 - The operating system of the DB instance is undergoing software patching
 - A manual failover of the DB instance was initiated using Reboot with failover

Read Replicas

You can reduce the load on your source DB instance by routing read queries from your applications to the Read Replica

Amazon RDS takes a snapshot of the source instance and creates a read-only instance from the snapshot

Amazon RDS then uses the asynchronous replication method for the DB engine to update the Read Replica whenever there is a change to the source DB instance

The Read Replica operates as a DB instance that allows only read-only connections.

Applications connect to a Read Replica the same way they do to any DB instance

you must enable automatic backups on the source DB instance

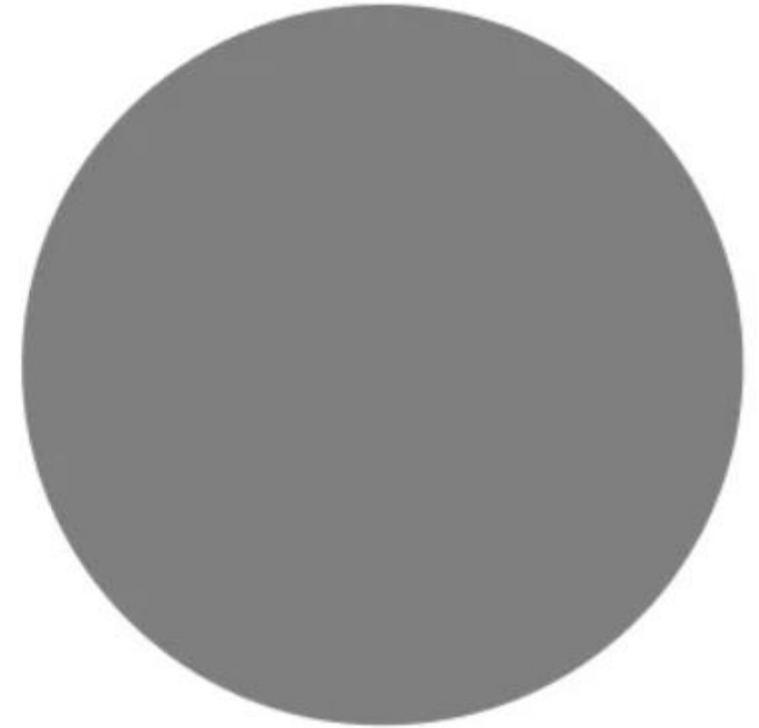
Read Replica Use cases

- Scaling beyond the compute or I/O capacity of a single DB instance for read-heavy database workloads
- Serving read traffic while the source DB instance is unavailable.
- Business reporting or data warehousing scenarios where you might want business reporting queries to run against a Read Replica

You can create a MySQL, PostgreSQL, or MariaDB Read Replica in a different AWS Region :

- Improve your disaster recovery capabilities
- Scale read operations into an AWS Region closer to your users
- Make it easier to migrate from a data center in one AWS Region to a data center in another AWS Region

Cross Region Replication



DB Parameter Group

You manage your DB engine configuration through the use of parameters in a DB parameter group

DB parameter groups act as a container for engine configuration values that are applied to one or more DB instances

A default DB parameter group is created if you create a DB instance without specifying a customer-created DB parameter group

This default group contains database engine defaults and Amazon RDS system defaults based on the engine, compute class, and allocated storage of the instance

Modifying Parameter Group

You cannot modify the parameter settings of a **default DB parameter group**

you must create your own DB parameter group to change parameter settings from their default value

When you change a **dynamic parameter** and save the DB parameter group, the change is applied immediately

When you change a **static parameter** and save the DB parameter group, the parameter change will take effect after you manually reboot the DB instance

When you change the DB parameter group associated with a DB instance, you must manually reboot the instance

Backup and Restore

- Amazon RDS creates a storage volume snapshot of your DB instance, backing up the entire DB instance and not just individual databases
- Amazon RDS saves the automated backups of your DB instance according to the backup retention period that you specify
- If necessary, you can recover your database to any point in time during the backup retention period
- You can also backup your DB instance manually, by manually creating a DB snapshot
- All automated backups are deleted when you delete a DB instance.
- Manual snapshots are not deleted

Backup Window

Automated backups occur **daily** during the preferred backup window

The backup window can't overlap with the weekly maintenance window for the DB instance

I/O activity is not suspended on your primary during backup for Multi-AZ deployments, because the backup is taken from the standby

If you don't specify a preferred backup window when you create the DB instance, Amazon RDS assigns a default 30-minute backup window

You can set the **backup retention period to between 1 and 35 days**

An outage occurs if you change the backup retention period from 0 to a non-zero value or from a non-zero value to 0

Monitoring

You can use the following automated monitoring tools to watch Amazon RDS and report when something is wrong:

- Amazon RDS Events
- Database log files
- Amazon RDS Enhanced Monitoring

RDS Security

Various ways you can secure RDS:

- Run your DB instance in an Amazon Virtual Private Cloud (VPC)
- Use AWS Identity and Access Management (IAM) policies to assign permissions that determine who is allowed to manage RDS resources
- Use security groups to control what IP addresses or Amazon EC2 instances can connect to your databases on a DB instance
- Use Secure Socket Layer (SSL) connections with DB instances
- Use RDS encryption to secure your RDS instances and snapshots at rest.
- Use the security features of your DB engine to control who can log in to the databases on a DB instance

DynamoDB



DynamoDB is a fully managed
NOSQL database , designed for
massive scale with predictable
performance goals



DynamoDB Features

- Every table in DynamoDB should be associated with a **primary key** (To be specified while creation)
- Any language of choice can be used to create , insert, update, query, scan(entire table) and delete operations on a dynamo table using appropriate API
- Each Row/record in a table is called an "**item**"
- DynamoDB allows to set **TTL** for individual items in a table to delete the items automatically on expiration
- The table data is stored in **SSD disks** and spread across multiple servers across different AZ in a region for faster performance, high availability and data durability
- The tables are **schema less**, except for the primary key , there is no requirements of the number and type of attributes
- DynamoDB offers encryption at rest

Read Consistency

DynamoDB supports eventually consistent and strongly consistent reads. DynamoDB uses eventually consistent reads, unless you specify otherwise.



Eventually Consistent Reads

When you read data from a DynamoDB table, the response might not reflect the results of a recently completed write operation. The response might include some stale data.



Strongly Consistent Reads

When you request a strongly consistent read, DynamoDB returns a response with the most up-to-date data, reflecting the updates from all prior write operations that were successful.

Throughput Capacity

- When you create a table or index in Amazon DynamoDB, you must specify your **capacity requirements for read and write activity**
- You specify throughput capacity in terms of read capacity units and write capacity units:
 - **One read capacity unit(RCU)** represents one strongly consistent read per second, or two eventually consistent reads per second, for an item up to 4 KB in size.
 - **One write capacity unit (WCU)** represents one write per second for an item up to 1 KB in size.

DynamoDB Autoscaling

DynamoDB auto scaling actively manages throughput capacity for tables and global secondary indexes.

With auto scaling, you define a range (upper and lower limits) for read and write capacity units.

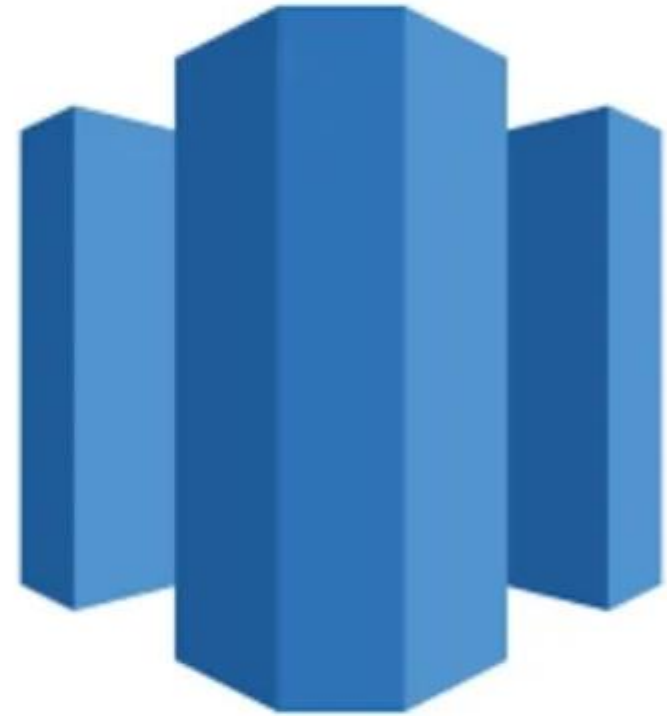
If you use the AWS Management Console to create a table or a global secondary index, DynamoDB auto scaling is enabled by default

You can manage auto scaling settings at any time by using the console, the AWS CLI, or one of the AWS SDKs.

AWS RedShift

AWS redshift is:

- **Simple**(to get started , to scale)
- **Fast** (Using the latest DW architectures) ,
- **Fully managed** (To patch , to backup and fault tolerant)
- **Petabyte scale** (Upto 2 PB) datawarehouse service
- **Based on PostGreSQL.**
- **Secure** (SSL on transit, encryption on rest , within VPC , no access to compute nodes)
- **Compatible with various industry BI tools** using JDBC/ODBC connectivity



AWS redshift uses **Massively parallel processing (MPP)** architecture, **columnar storage**, **data compression** and **zone mapping** for faster **query performance** on data sets.

Hardware is optimized for large data processing with features of **locally attached storage devices** , **10gig mesh network** and **1 MB of block size**

There are two types of nodes that can be selected in a redshift cluster

1) DS2 node types are optimized for large data workloads and use hard disk drive (HDD) storage, 2) DC2 nodes uses SSD disks

Node size and the number of nodes determine the total storage for a cluster

All the cluster nodes are created in the same AZ of a region

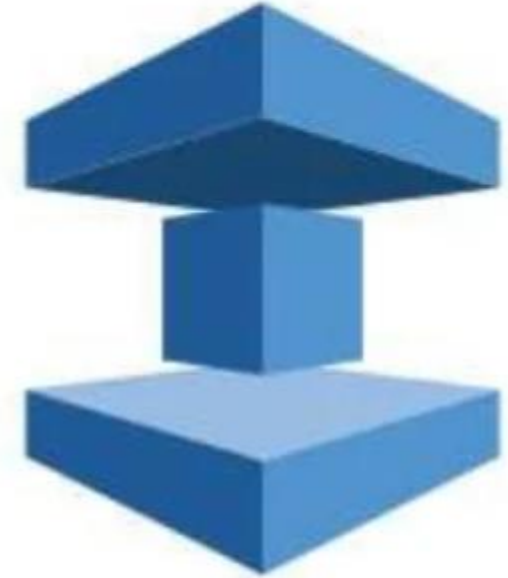
There are two types of monitoring metrics produced every minute ie 1) cloud watch metrics 2) query performance metrics which is not published to cloudwatch

The automated snapshots backups are taken usually every 8 hours or every 5 GB of data change

RedShift Features

Elasticache

- Elasticache is a distributed memory cache system / data store
- There are two engines supported : **Redis** , **Memcached**
- Three main methods of how to cache: **Lazy Population**, **Write through** , **Timed refresh (TTL)**



Memcached

Memcached is a "Gold Standard"

Memcached is simple to use , multithreaded

Memcached clusters are made of 1 to 20 nodes and maximum 100 nodes in a region

Horizontal scaling in Memcached is easy and it is just about adding or removing the nodes

Vertical scaling in Memcached would create a new cluster with empty data

Backup / restore capability and replication features are available only with Redis

Redis

Redis is single-threaded

Redis has two flavors : Cluster mode disabled(Only one shard) and cluster mode enabled (one to 15 shards)

A Redis Shard (node group) can have 1 to 6 nodes with the replication option of one node primary and other read replicas

Read replicas of Redis are synced asynchronously

Multi-AZ with Autorecovery is enabled by default for Redis cluster with cluster mode enabled

Backups are stored in S3 with 0 to 35 days retention period.

Deployment and Management Services

IAM

CloudWatch

CloudTrail

CloudFormation

SNS

KMS

CloudConfig

IAM

- IAM Features
- How IAM works? Infrastructure Elements
- Identities
- Access Management
- IAM Best Practices



You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources.

When you first create an AWS account, you begin with a single sign-in identity that has complete access to all AWS services and resources in the account.

This identity is called the AWS account *root user* and is accessed by signing in with the email address and password that you used to create the account

Identity and Access Management (IAM)

IAM Features

1. Shared access to your AWS account
2. Granular permissions
3. Secure access to AWS resources for applications that run on Amazon EC2
4. Multi-factor authentication (MFA)
5. Identity federation
6. Identity information for assurance
7. PCI DSS Compliance
8. Integrated with many AWS services
9. Eventually Consistent
10. Free to use

How IAM Works: IAM Infrastructure Elements

1. Principal

2. Request

3.
Authentication

4.
Authorization

5. Actions

6. Resources

Principal

A *principal* is an entity that can take an action on an AWS resource. AWS Users, roles, federated users, and applications are all AWS principals.

Request

When a principal tries to use the AWS Management Console, the AWS API, or the AWS CLI, that principal sends a *request* to AWS. A request specifies the following information:

- Actions (or operations) that the principal wants to perform
- Resources upon which the actions are performed
- Principal information, including the environment from which the request was made

AWS gathers this information into a *request context*, which is used to evaluate and authorize the request.

Authentication

As a principal, you must be **authenticated (signed in to AWS)** to send a request to AWS.

Alternatively, a few services, like Amazon S3, allow requests from anonymous users

To authenticate from the console, you must sign in with your user name and password.

To authenticate from the API or CLI, you must provide your **access key and secret key**.

AWS recommends that you use **multi-factor authentication (MFA)** to increase the security of your account.

Authorization

- During authorization, IAM uses values from the *request context* to check for *matching policies* and determine whether to allow or deny the request.
- Policies are stored in IAM as *JSON documents* and specify the permissions that are allowed or denied for principals
- If a single policy includes a denied action, IAM denies the entire request and *stops evaluating*. This is called an *explicit deny*.
- The evaluation logic follows these rules:
 - *By default, all requests are denied.*
 - An explicit allow overrides this default.
 - *An explicit deny overrides any allows.*

After your request has been *authenticated* and *authorized*, **AWS approves the actions** in your request.

Actions are defined by a service, and are the things that you can do to a resource, such as viewing, creating, editing, and deleting that resource.

For example, *IAM supports around 40 actions* for a user resource, including the following actions:

- Create User
- Delete User
- GetUser
- UpdateUser

Actions

Resources

A resource is an entity that exists within a service. Examples include an *Amazon EC2 instance*, an *IAM user*, and an *Amazon S3 bucket*.

After AWS approves the actions in your request, those actions can be performed on the related resources within your account..

You create *IAM Identities* to provide authentication for people and processes in your AWS account.

- IAM Users
- IAM Groups
- IAM Roles

IAM Identities



IAM Users

The IAM user represents the person or service who uses the IAM user to interact with AWS.

When you create a user, IAM creates these ways to **identify that user**:

- A "**friendly name**" for the user, which is the name that you specified when you created the user, such as Bob or Alice. These are the names you see in the AWS Management Console
- An **Amazon Resource Name (ARN)** for the user. You use the ARN when you need to uniquely identify the user across all of AWS, such as when you specify the user as a Principal in an IAM policy for an Amazon S3 bucket. An ARN for an IAM user might look like the following:
arn:aws:iam::account-ID-without-hyphens:user/Bob
- A unique identifier for the user. This ID is returned only when you use the API, Tools for Windows PowerShell, or AWS CLI to create the user; you do not see this ID in the console

IAM Groups

An IAM group is a collection of IAM users. You can use groups to specify permissions for a collection of users, which can make those permissions easier to manage for those users

Following are some important characteristics of groups:

A group can contain many users, and a user can belong to multiple groups

Groups can't be nested; they can contain only users, not other groups.

There's no default group that automatically includes all users in the AWS account.

There's a limit to the number of groups you can have, and a limit to how many groups a user can be in.

IAM Roles

- An IAM role is very similar to a user, However, a **role does not have any credentials** (password or access keys) associated with it.
- Instead of being uniquely associated with one person, a role is intended to be **assumable by anyone** who needs it
- If a user assumes a role, **temporary security credentials** are created dynamically and provided to the user.
- Roles can be used by the following:
 - An IAM user in the same AWS account as the role
 - An IAM user in a different AWS account as the role
 - A web service offered by AWS such as Amazon Elastic Compute Cloud (Amazon EC2)
 - An external user authenticated by an external identity provider (IdP) service that is compatible with SAML 2.0 or OpenID Connect, or a custom-built identity broker

IAM User vs Role

When to Create an IAM User (Instead of a Role):

- You created an AWS account and you're the only person who works in your account.
- Other people in your group need to work in your AWS account, and your group is using no other identity mechanism.
- You want to use the command-line interface (CLI) to work with AWS.

When to Create an IAM Role (Instead of a User) :

- You're creating an application that runs on an Amazon Elastic Compute Cloud (Amazon EC2) instance and that application makes requests to AWS
- You're creating an app that runs on a mobile phone and that makes requests to AWS.
- Users in your company are authenticated in your corporate network and want to be able to use AWS without having to sign in again—that is, you want to allow users to federate into AWS.

Access Management

When a principal makes a request in AWS, the IAM service checks whether the principal is authenticated (signed in) and authorized (has permissions)

You manage access by creating policies and attaching them to IAM identities or AWS resources

Policies

- Policies are stored in AWS as JSON documents attached to principals as **identity-based policies**, or to resources as **resource-based policies**
- A policy consists of one or more statements, each of which describes one set of permissions.
- Here's an example of a simple policy.


```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::example_bucket"
  }
}
```

Identity Based Policy

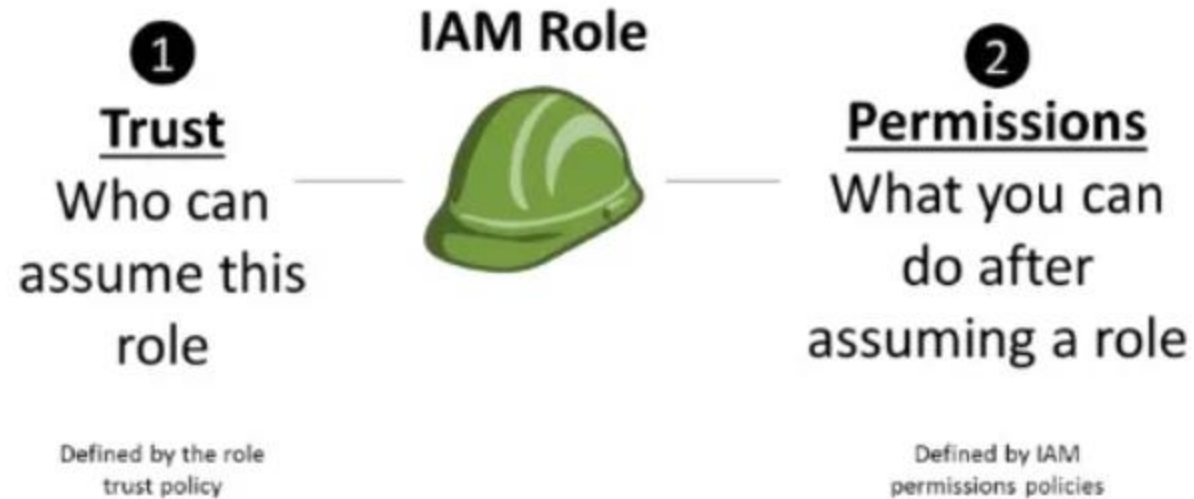
- Identity-based policies are permission policies that you can attach to a principal (or identity), such as an IAM user, role, or group.
- These policies control what actions that identity can perform, on which resources, and under what conditions.
- Identity-based policies can be further categorized:
 - **Managed policies** – Standalone identity-based policies that you can attach to multiple users, groups, and roles in your AWS account. You can use two types of managed policies:
 - **AWS managed policies** – Managed policies that are created and managed by AWS. If you are new to using policies, we recommend that you start by using AWS managed policies
 - **Customer managed policies** – Managed policies that you create and manage in your AWS account. Customer managed policies provide more precise control over your policies than AWS managed policies.
 - **Inline policies** – Policies that you create and manage and that are embedded directly into a single user, group, or role.

Resource-based policies are JSON policy documents that you attach to a resource such as an Amazon S3 bucket.

These policies control what actions a specified principal can perform on that resource and under what conditions

Resource-based policies are inline policies, and there are no managed resource-based policies.

Resource Based Policies



- Trust policies are resource-based policies that are **attached to a role** that define which principals can assume the role.
- When you create a role in IAM, the role must have two things: The first is a trust policy that indicates who can assume the role. The second is a permission policy that indicates what they can do with that role

Trust Policies

Top 11 IAM best practices

0. **Users** – Create individual users.
1. **Permissions** – Grant least privilege.
2. **Groups** – Manage permissions with groups.
3. **Conditions** – Restrict privileged access further with conditions.
4. **Auditing** – Enable AWS CloudTrail to get logs of API calls.
5. **Password** – Configure a strong password policy.
6. **Rotate** – Rotate security credentials regularly.
7. **MFA** – Enable MFA for privileged users.
8. **Sharing** – Use IAM roles to share access.
9. **Roles** – Use IAM roles for Amazon EC2 instances.
10. **Root** – Reduce or remove use of root.

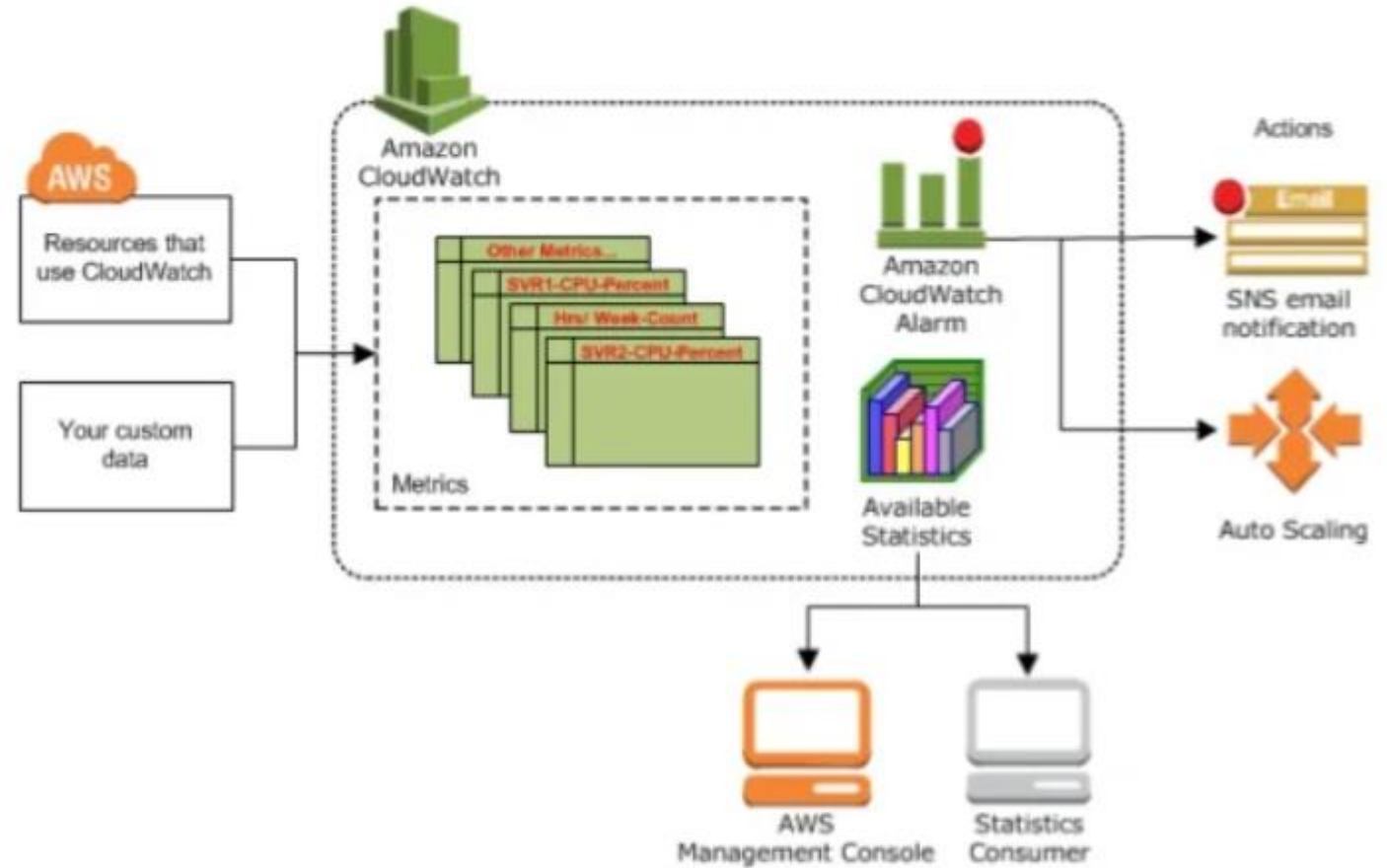
CloudWatch

- Introduction
- Concepts
- Dashboard
- CloudWatch Agent



Introduction

- Amazon CloudWatch is basically a **metrics repository**
- An AWS service—such as Amazon EC2—puts metrics into the repository, and you retrieve statistics based on those metrics
- If you put your own custom metrics into the repository, you can retrieve statistics on these metrics as well



Concepts

Namespaces

Metrics

Dimensions

Statistics

Alarms

Metrics

- Metrics are the **fundamental concept** in CloudWatch
- For example, the CPU usage of a particular EC2 instance is one metric provided by Amazon EC2
- A metric represents a time-ordered set of data points that are published to CloudWatch
- You can add the **data points** in any order, and at any rate you choose
- Metrics exist only in the region in which they are created
- **Metrics cannot be deleted**, but they automatically **expire after 15 months** if no new data is published to them
- Data points older than 15 months expire on a rolling basis
- Metrics are uniquely defined by a name, a namespace, and zero or more dimensions

Metrics Retention



- Data points with a period of less than 60 seconds are available for 3 hours. These data points are high-resolution custom metrics
- Data points with a period of 60 seconds (1 minute) are available for 15 days
- Data points with a period of 300 seconds (5 minute) are available for 63 days
- Data points with a period of 3600 seconds (1 hour) are available for 455 days (15 months)

Namespaces

A namespace is a container for CloudWatch metrics

Metrics in different namespaces are isolated from each other, so that metrics from different applications are not mistakenly aggregated into the same statistics

There is no default namespace. You must specify a namespace for each data point you publish to CloudWatch

The AWS namespaces use the following naming convention: AWS/service. For example, Amazon EC2 uses the AWS/EC2 namespace.

Dimension

A dimension is a name/value pair that uniquely identifies a metric. You can assign up to 10 dimensions to a metric.

Every metric has specific characteristics that describe it, and you can think of dimensions as categories for those characteristics

AWS services that send data to CloudWatch attach dimensions to each metric.

You can use dimensions to filter the results that CloudWatch returns

For example, you can get statistics for a specific EC2 instance by specifying the InstanceId dimension when you search for metrics

Statistics

- Statistics are metric data aggregations over specified periods of time
- Aggregations are made using the namespace, metric name, dimensions, and the data point unit of measure, within the time period you specify
- Various statistics include "minimum", "maximum", "Sum", "Average".
- Each statistic has a unit of measure. Example units include Bytes, Seconds, Count, and Percent.
- A period is the length of time associated with a specific Amazon CloudWatch statistic
- A period can be as short as one second or as long as one day (86,400 seconds). The default value is 60 seconds

Alarms

- You can use an alarm to automatically initiate actions on your behalf
- An alarm watches a single metric over a specified time period, and performs one or more specified actions, based on the value of the metric relative to a threshold over time
- The action is a **notification sent to an Amazon SNS topic or an Auto Scaling policy**. You can also add alarms to dashboards
- Alarms invoke actions for sustained state changes only. The state must have changed and been maintained for a specified number of periods

Dashboard

Amazon CloudWatch dashboards are customizable home pages in the CloudWatch console

You can use to monitor your resources in a single view, even those resources that are spread across different regions.

You can use CloudWatch dashboards to create customized views of the metrics and alarms for your AWS resources

The unified CloudWatch agent enables you to do the following:

- Collect more system-level metrics from Amazon EC2 instances in addition to the metrics listed in Amazon EC2 Metrics and Dimensions
- Collect system-level metrics from on-premises servers. These can include servers in a hybrid environment as well as servers not managed by AWS
- Collect logs from Amazon EC2 instances and on-premises servers, running either Linux or Windows Server

CloudWatch Agents

Summary : CloudWatch

- ☐ CloudWatch is a metrics repository , you can retrieve statistics based on these metrics
- ☐ Metrics are uniquely defined by a name, a namespace, and zero or more dimensions
- ☐ An alarm watches a single metric over a specified time period, and performs one or more specified actions, based on the value of the metric
- ☐ CloudWatch agents allows to monitor on premise servers and generate custom metrics



AWS CloudTrail

AWS CloudTrail service enable governance, compliance, and operational and risk auditing of your AWS account.

Overview

Actions taken by a user, role, or an AWS service are recorded as events in CloudTrail

Events include actions taken in the AWS Management Console, AWS Command Line Interface, and AWS SDKs and APIs.

You can identify who or what took which action, what resources were acted upon, when the event occurred

CloudTrail is enabled on your AWS account when you create it.

You can easily view events of past 90 days in the CloudTrail console by going to Event history.

Trail

- In addition, you can create a CloudTrail **trail** to archive, analyze, and respond to changes in your AWS resources
- A trail is a configuration that enables delivery of events to an Amazon S3 bucket that you specify
- You can also deliver and analyze events in a trail with Amazon CloudWatch Logs and Amazon CloudWatch Events
- By default CloudTrail log stored in S3 is encrypted with SSE
- Cloudtrail delivers two types of events : 1 management events eg create bucket , create S3 bucket
2) data events Eg S3 object API
- By default, trails log management events, but not data events.
- CloudTrail does not log all AWS services. Some AWS services do not enable logging of all APIs and events.

Cloud Formation

- CloudFormation is used to create a reusable templates which be used to automate the deployment of the AWS resource
- When you use AWS CloudFormation, you work with templates and stacks.
- You create templates to describe your AWS resources and their properties.
- Whenever you create a stack, AWS CloudFormation provisions the resources that are described in your template.



CloudFormation

CloudFormation templates

An AWS CloudFormation template is a JSON or YAML formatted text file.

AWS CloudFormation uses these templates as blueprints for building your AWS resources.

Templates would have the below 9 sections:

- Templateformatversion (Optional section)
- Description (Optional Section)
- Parameters (Optional section)
- Mappings (Optional)
- Conditions (Options)
- **Resources** (Required Section)
- Outputs (Optional)
- Transform (Optional)
- Metadata (Optional)

Only "Resources" section is mandatory

Example JSON template



```
{  
  "AWSTemplateFormatVersion" : "2010-09-09",
```

```
  "Description" : "A sample template",
```

```
  "Resources" : {
```

```
    "MyEC2Instance" : {
```

```
      "Type" : "AWS::EC2::Instance",
```

```
      "Properties" : {
```

```
        "ImageId" : "ami-2f726546",
```

```
        "InstanceType" : "t1.micro",
```

```
        "KeyName" : "testkey",
```

```
        "BlockDeviceMappings" : [
```

```
          {
```

```
            "DeviceName" : "/dev/sdm",
```

```
            "Ebs" : {
```

```
              "VolumeType" : "io1",
```

```
              "Iops" : "200",
```

```
              "DeleteOnTermination" : "false",
```

```
              "VolumeSize" : "20"
```

```
          ]
```

```
        }
```

```
      }
```

```
    }
```

When you use AWS CloudFormation, you manage related resources as a single unit called a stack.

All the resources in a stack are defined by the stack's AWS CloudFormation template.

CloudFormation can perform only actions that you have permission to do.

When you delete a stack, AWS CloudFormation deletes the stack and all the resources in that stack.

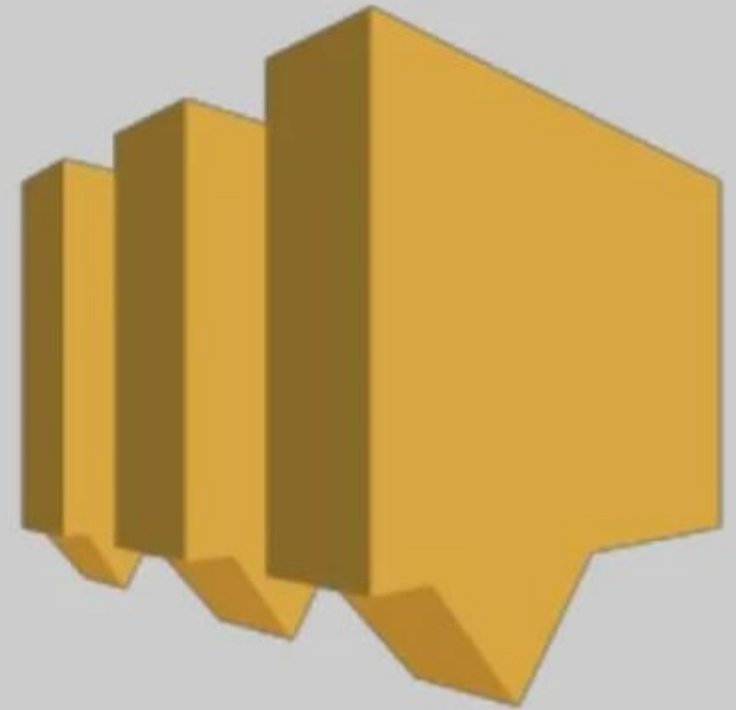
If you want to delete a stack but want to retain some resources in that stack, you can use a deletion policy to retain those resources.

A graphic at the bottom of the slide featuring two overlapping trapezoidal shapes. The left shape is light gray and contains the text 'CloudFormation stack'. The right shape is a darker gray and is positioned behind the first one.

CloudFormation stack

SNS

- Amazon Simple Notification Service coordinates and manages the delivery or sending of messages to subscribing endpoints or clients



Overview

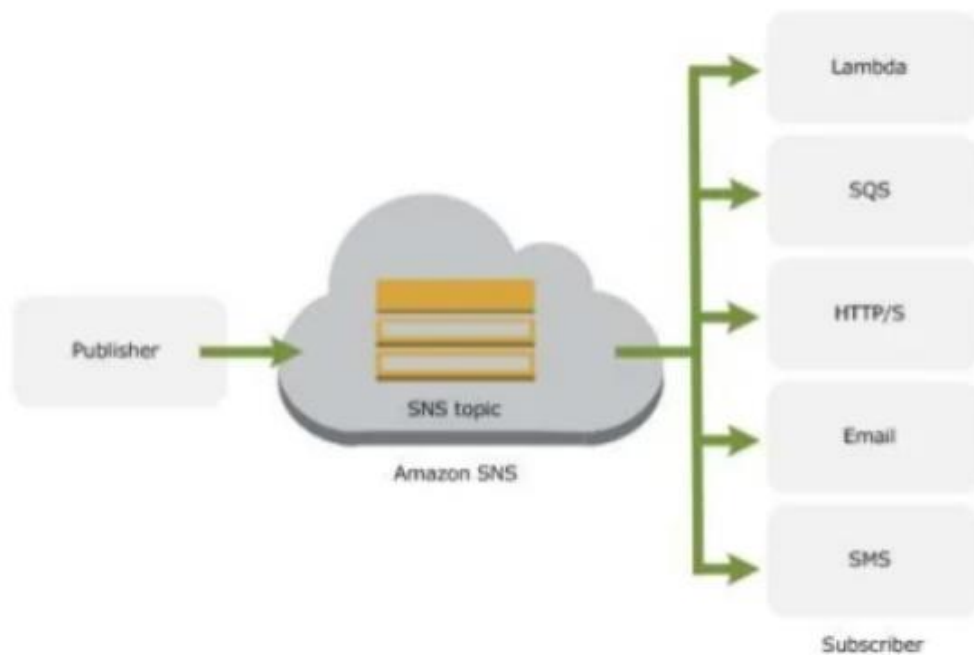
In Amazon SNS, there are two types of clients—publishers and subscribers —also referred to as producers and consumers.

Publishers communicate asynchronously with subscribers by producing and sending a message to a **topic**, which is a logical access point and communication channel

SNS is used to either notify certain users on certain alerts or to automate by integrating multiple components of AWS by notifying between the AWS services

SNS uses "push" method to handle the messages unlike SQS which uses "poll" ing

When using Amazon SNS, you (as the owner) create a topic and control access to it by defining policies that determine which publishers and subscribers can communicate with the topic.



How it works

- A publisher sends messages to topics that they have created or to topics they have permission to publish to
- Instead of including a specific destination address in each message, a publisher sends a message to the topic
- Amazon SNS matches the topic to a list of subscribers who have subscribed to that topic, and delivers the message to each of those subscribers.
- Subscribers receive all messages published to the topics to which they subscribe, and all subscribers to a topic receive the same messages.



KMS

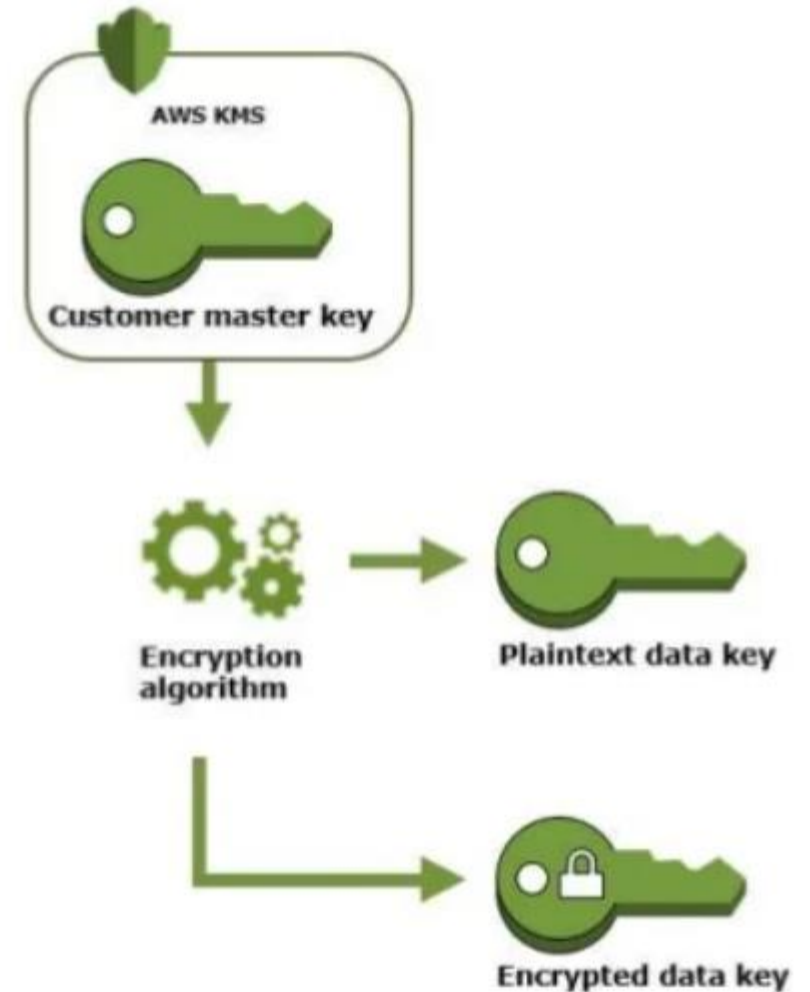
AWS Key Management Service (AWS KMS) is a managed service that makes it easy for you to create and control the encryption keys used to encrypt your data.

Customer Master Key

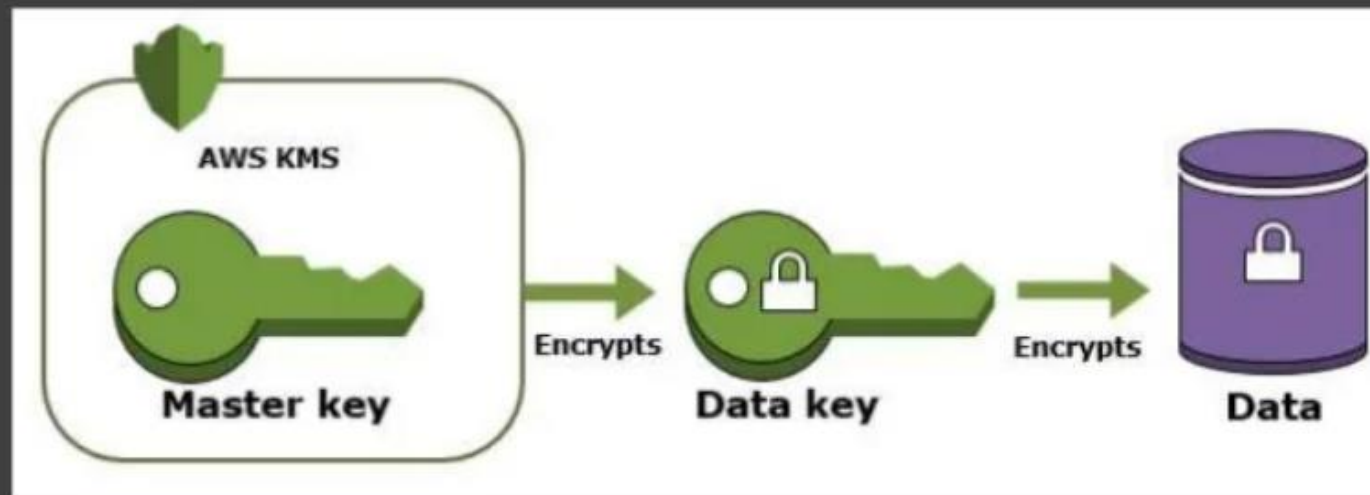
- The primary resources in AWS KMS are customer master keys (CMKs).
- You can use a CMK to encrypt and decrypt up to 4 kilobytes (4096 bytes) of data.
- AWS KMS stores, tracks, and protects your CMKs.
- When you want to use a CMK, you access it through AWS KMS. CMKs never leave AWS KMS unencrypted.
- Each CMK is secured primarily by KMS access policies and must have one
- AWS makes it mandatory to give waiting period between 7 to 30 days before deletion to prevent unintentional deletion.
- There are two types of CMKs in AWS accounts:
 - Customer managed CMKs are CMKs that you create, manage, and use.
 - AWS managed CMKs are CMKs in your account that are created, managed, and used on your behalf by an AWS service like EBS that is integrated with AWS KMS

Data Keys

- Data keys are encryption keys that you can use to encrypt data, including large amounts of data and other data encryption keys
- You can use AWS KMS customer master keys (CMKs) to generate, encrypt, and decrypt data keys.
- However, AWS KMS does not store, manage, or track your data keys, or perform cryptographic operations with data keys.
- You must use and manage data keys outside of AWS KMS.



Envelope Encryption Keys



- Envelope encryption is the practice of encrypting plaintext data with a data key, and then encrypting the data key under another key.
- You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key another encryption key.

AWS Config

AWS Config provides a detailed view of the configuration of AWS resources in your AWS account.



Overview

With AWS Config, you can do the following:

- Evaluate your AWS resource configurations for desired settings.
- Get a snapshot of the current configurations of the supported resources that are associated with your AWS account
- Retrieve configurations of one or more resources that exist in your account
- Retrieve historical configurations of one or more resources.
- Receive a notification whenever a resource is created, modified, or deleted.
- View relationships between resources. For example, you might want to find all resources that use a particular security group



Use Cases

An AWS Config rule represents your desired configuration settings for specific AWS resources or for an entire AWS account

AWS Config provides customizable, predefined rules to help you get started.

You can also create custom rules using AWS Lambda

If a resource violates a rule, AWS Config flags the resource and the rule as noncompliant, and AWS Config notifies you through Amazon SNS.

AWS Config Rules

How AWS config works?

When you turn on AWS Config, it first discovers the supported AWS resources that exist in your account and generates a configuration item for each resource.

AWS Config also generates configuration items when the configuration of a resource changes

It maintains historical records of the configuration items of your resources from the time you start the configuration recorder.

AWS Config keeps track of all changes to your resources by invoking the Describe or the List API call for each resource in your account.

AWS Trusted Advisor

An online resource to help you reduce cost,
increase performance, and improve security by
optimizing your AWS environment



Overview

Like your customized cloud expert, AWS Trusted Advisor analyzes your AWS environment and provides best practice recommendations in five categories:



Cost Optimization



Performance



Security



Fault Tolerance



Service Limits

Application and Other Services

SQS

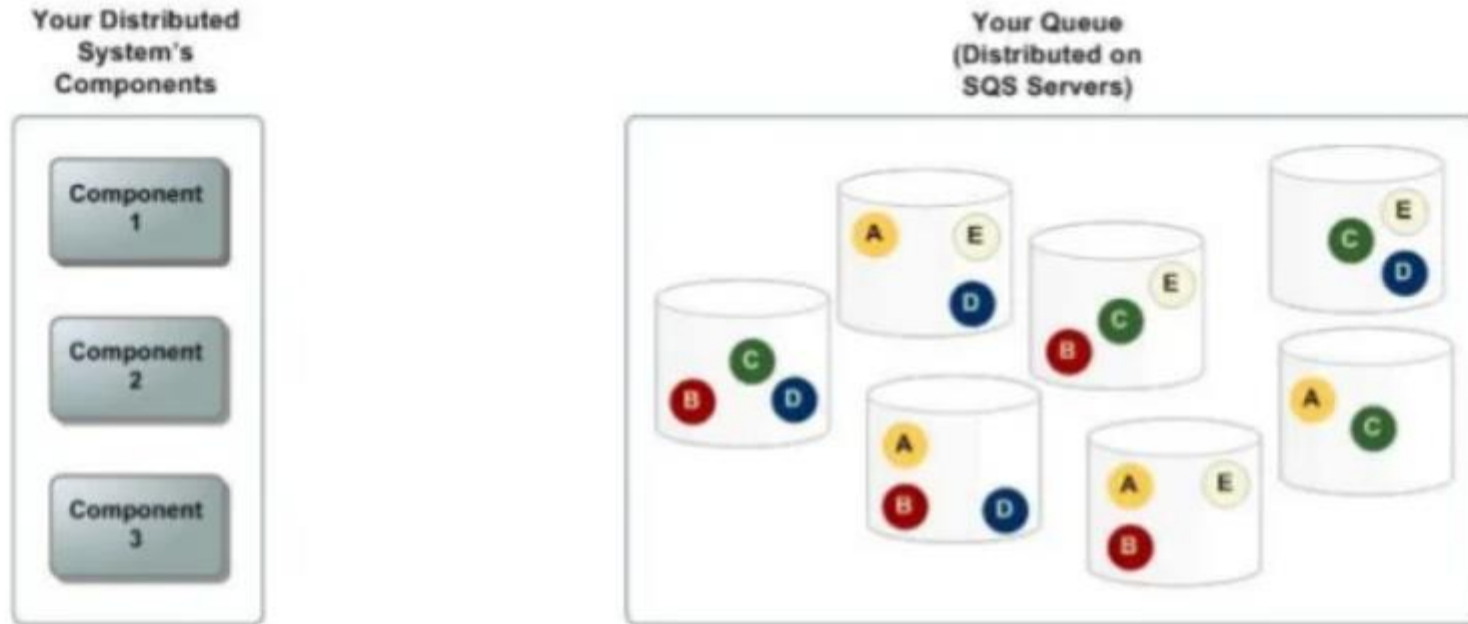
EMR

Kinesis

Amazon Simple Queue Service (Amazon SQS) offers a secure, durable, and available hosted queue that lets you integrate and decouple distributed software systems and components.

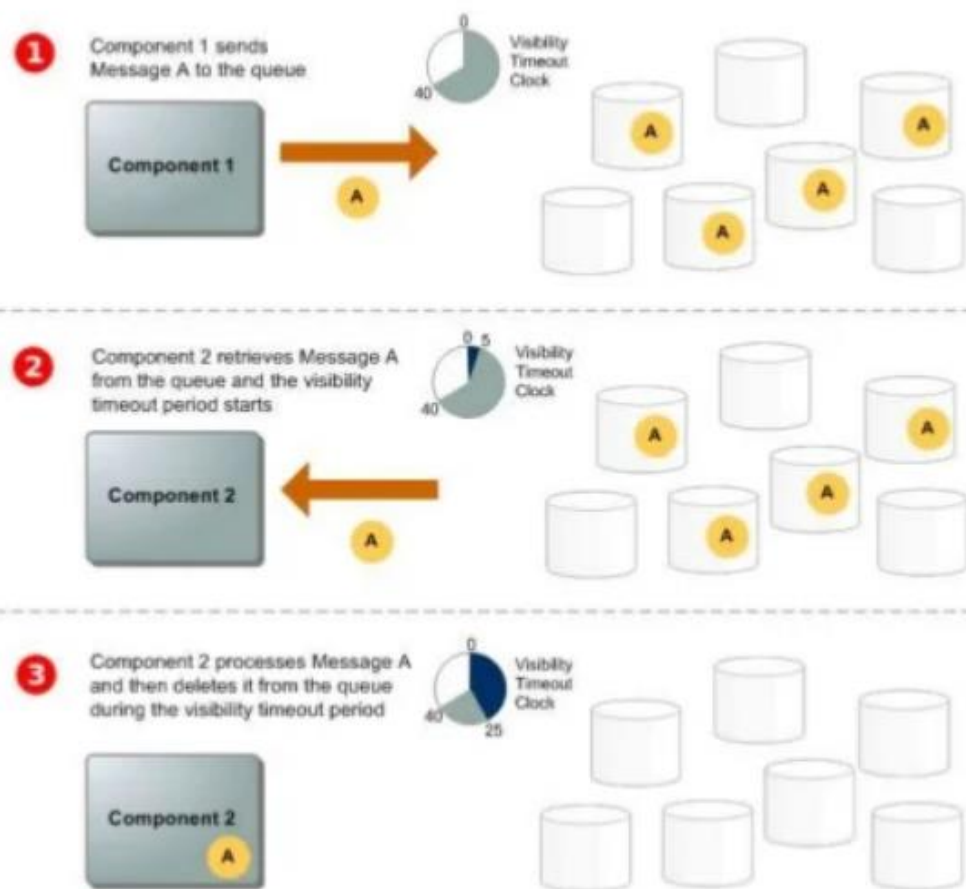
SQS





Distributed queues

There are three main parts in a distributed messaging system: the components of your distributed system, your queue (distributed on Amazon SQS servers), and the messages in the queue



Message Life cycle

- A **producer** (component 1) sends message A to a queue, and the message is distributed across the Amazon SQS servers redundantly.
- When a **consumer** (component 2) is ready to process messages, it consumes messages from the queue, and message A is returned.
- While message A is being processed, it remains in the queue and isn't returned to subsequent receive requests for the duration of the [visibility timeout](#).
- The consumer (component 2) deletes message A from the queue to prevent the message from being received and processed again when the visibility timeout expires



Kinesis

Amazon Kinesis makes it easy to collect, process, and analyze video and data streams in real time.

Stream types

1

Use **Kinesis Video Streams** to capture, process, and store video streams for analytics and machine learning

2

Use **Kinesis Data Streams** to build custom applications that analyze data streams using popular stream processing frameworks.

3

Use **Kinesis Data Firehose** to load data streams into AWS data stores.

4

Use **Kinesis Data Analytics** to analyze data streams with SQL.

EMR

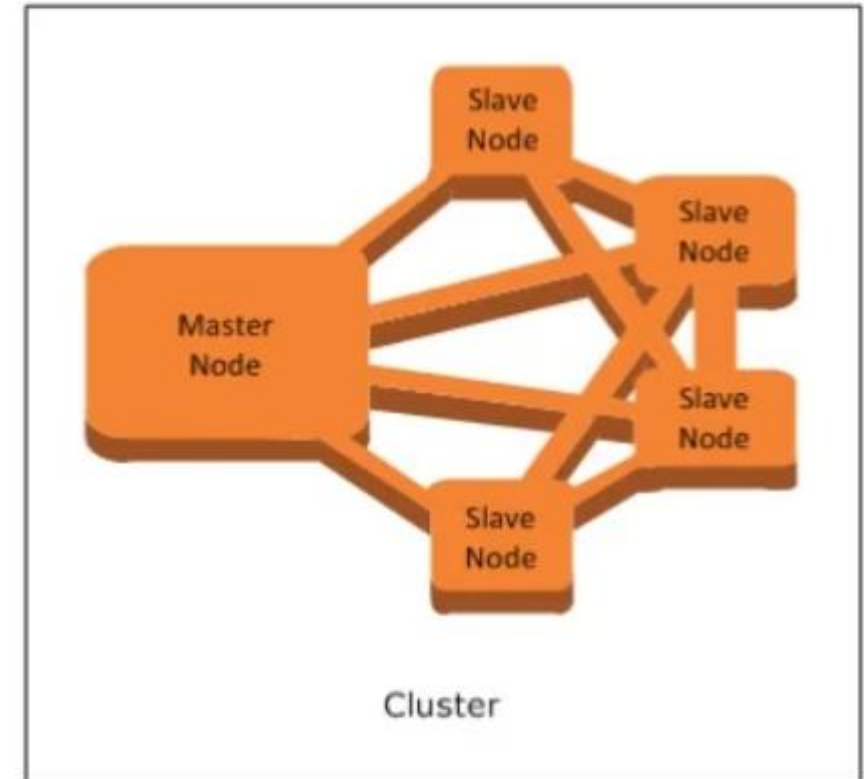
Amazon EMR is a managed cluster platform that simplifies running big data frameworks, such as Apache Hadoop and Apache Spark, on AWS to process and analyze vast amounts of data.



Cluster nodes

The node types in Amazon EMR Cluster are as follows:

- **Master node:** a node that manages the cluster by running software components which coordinate the distribution of data and tasks among other nodes—collectively referred to as slave nodes
- **Core node:** a slave node that has software components which run tasks and store data in the Hadoop Distributed File System (HDFS) on your cluster
- **Task node:** a slave node that has software components which only run tasks. Task nodes are optional.



Cluster Storage Options

Hadoop Distributed File System (HDFS)

- Distributes the data it stores across instances in the cluster, storing multiple copies of data on different instances to ensure that no data is lost if an individual instance fails
- HDFS is ephemeral storage that is reclaimed when you terminate a cluster.
- HDFS is useful for caching intermediate results during MapReduce processing or for workloads which have significant random I/O

EMR File System (EMRFS)

- EMR File System (EMRFS), Amazon EMR extends Hadoop to add the ability to directly access data stored in Amazon S3 as if it were a file system like HDFS.
- EMRFS provides the advantage of loosely coupled architecture between compute and storage where we don't need to scale instances just because of storage expansion
- EMRFS also provides the advantage of persistent storage and the cluster can be shutdown when not in use

Local File System

- The local file system refers to a locally connected disk. Data on instance store volumes persists only during the lifecycle of its Amazon EC2 instance.